UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

CORSO DI LAUREA IN INGEGNERIA INFORMATICA
CORSO DI INGEGNERIA DEL SOFTWARE
PROF. A.R. FASOLINO - A.A. 2025-26...



PROGETTO
DI INGEGNERIA DEL SOFTWARE - GRUPPO 8

"Sistema di Registro Elettronico di Classe"

INDICE

1. SPECIFICHE INFORMALI 1

2. ANALISI E SPECIFICA DEI REQUISITI 2

2.1 ANALISI NOMI-VERBI 2
2.2 REVISIONE DEI REQUISITI 3
2.3 GLOSSARIO DEI TERMINI 4
2.4 CLASSIFICAZIONE DEI REQUISITI 5
2.4.1 Requisiti funzionali 5
2.4.2 Requisiti sui dati 6
2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010) 7
2.4.4 Vincoli / Altri requisiti 9
2.5 MODELLAZIONE DEI CASI D'USO 10
2.5.1 Attori e casi d'uso 10
2.5.2 Diagramma dei casi d'uso 12
2.5.3 Scenari 12
2.6 DIAGRAMMA DELLE CLASSI 16
2.7 DIAGRAMMI DI SEQUENZA 20
2.7.1 Creazione classe virtuale 20
2.7.2 Iscrizione tramite codice classe 20
2.7.3 Registra valutazione 21
2.7.4 Visualizza valutazioni singolo studente 22
2.8 DIAGRAMMA DELLE CLASSI RAFFINATO 24

3. PIANO DI TEST FUNZIONALE 26

3.1 CREA CLASSE VIRTUALE 26
3.2 ISCRIZIONE TRAMITE CODICE CLASSE 27
3.3 REGISTRA VALUTAZIONE 28

4. PROGETTAZIONE 31

4.1 WIREFRAMES 31
4.1.1 Schermata Iniziale 31
4.1.2 Schermata Studente 32
4.1.3 Schermata Docente 33
4.1.4 Schermata Classe 34
4.1.5 Schermata Registro 35
4.1.6 Schermata Seleziona Registra Voce 36
4.1.7 Schermata Registra Valutazione 37
4.1.8 Schermata Visualizza Info 38
4.1.9 Schermata Valutazioni Studenti 39
4.2 DIAGRAMMA DELLE CLASSI 40
4.2.1 Traduzione classi ed associazioni 40
4.2.2 Pattern BCED 42
4.2.2.1 Package Boundary 42
4.2.2.2 Package Controller 43
4.2.2.3 Package Entity 45
4.2.2.4 Package Database 48
4.2.2.5 Package Exceptions 49
4.2.2.6 Package Comunicazione 50
4.3 DIAGRAMMI DI SEQUENZA 51
4.3.1 Creazione classe virtuale 51
4.3.2 Iscrizione tramite codice classe 52
4.3.3 Registra Valutazione 53
4.3.4 Visualizza valutazioni singolo studente 54

5. IMPLEMENTAZIONE 55

5.1 PACKAGE DATABASE 55
5.2 PACKAGE ENTITY 55

5.3 PACKAGE CONTROLLER 60
5.4 PACKAGE BOUNDARY 60
5.5 PACKAGE DTO 61
5.6 PACKAGE EXCEPTIONS 61
5.7 PACKAGE COMUNICAZIONE 62
5.8 ARTEFATTI NECESSARI 62
5.9 DIAGRAMMA DI DEPLOYMENT 63

6. TESTING 64

6.1 TEST STRUTTURALE 64
6.1.1 Complessità ciclomatica 64
6.1.1.1 verificaUnivocitaNome – Docente 64
6.1.1.2 cercaValutazioniStudente – Registro 65
6.2 JUNIT – TEST DI UNITÀ 67
6.3 TEST FUNZIONALE 68

1. Specifiche informali

Riportare la Traccia assegnata

Si desidera sviluppare un sistema software per la gestione di un registro elettronico di classe, finalizzato alla registrazione delle lezioni svolte, dei compiti assegnati e delle valutazioni ottenute dagli studenti nel corso delle attività didattiche. Il sistema dovrà consentire ai docenti di gestire le informazioni relative alle proprie classi e agli studenti di consultare i dati di proprio interesse. Descrizione del Sistema: Il sistema consente l'accesso di docenti e studenti a una piattaforma digitale mediante autenticazione tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo (studente o docente), oltre a fornire nome, cognome ed indirizzo email istituzionale. Ogni docente ha la possibilità di creare e gestire una o più classi virtuali, ciascuna identificata da un nome e da un codice univoco. L'iscrizione di uno studente a una classe può avvenire in due modalità: il docente può iscrivere direttamente lo studente alla classe, oppure lo studente stesso può iscriversi autonomamente inserendo il codice univoco della classe fornito dal docente. All'interno di ogni classe, il docente può registrare le lezioni svolte. Per ciascuna lezione devono essere specificati almeno la data, l'argomento trattato ed eventualmente una breve descrizione aggiuntiva. Le lezioni devono essere visualizzabili in ordine cronologico all'interno del registro della classe. Il docente può inoltre assegnare compiti o attività da svolgere a casa. Ogni compito è definito da un titolo, una descrizione, una data di assegnazione e, opzionalmente, una data di scadenza. Gli studenti possono consultare l'elenco dei compiti assegnati direttamente all'interno del proprio profilo personale, con l'indicazione della classe e della data associata. Il sistema consente al docente di registrare le valutazioni ottenute dagli studenti relativamente a prove scritte e interrogazioni orali. Per ciascuna valutazione devono essere indicati almeno lo studente a cui si riferisce, la data, la tipologia della prova, il voto assegnato ed eventualmente una breve nota descrittiva. Le tipologie di prova considerate dal sistema sono almeno due: "prova scritta" e "interrogazione". Ogni studente può visualizzare, all'interno del proprio profilo personale, l'elenco dei voti ricevuti per ciascuna classe, con indicazione della data, della tipologia di prova e del valore ottenuto. Deve inoltre essere possibile consultare una media dei voti relativa alla singola classe, calcolata sulla base delle valutazioni registrate dal docente. Il docente, attraverso una interfaccia dedicata, può visualizzare il registro completo di ciascuna classe, comprendente l'elenco delle lezioni svolte, dei compiti assegnati e delle valutazioni registrate. Deve inoltre essere possibile accedere al dettaglio delle informazioni relative a ciascuno studente, visualizzando lo storico dei voti ricevuti e l'elenco dei compiti assegnati alla classe di appartenenza. Il sistema mantiene lo storico di tutte le attività svolte per ciascuna classe. Gli studenti possono consultare l'elenco delle lezioni già svolte, dei compiti assegnati e dei voti ricevuti. I docenti, attraverso un'interfaccia dedicata, possono monitorare l'andamento complessivo della classe, visualizzando il numero di lezioni registrate, il numero di compiti assegnati, la media generale della classe e il numero totale di valutazioni inserite in un determinato intervallo di tempo. Per semplicità, si può supporre che ogni classe sia gestita da un singolo docente e che ogni voto sia espresso in forma numerica. Il sistema include inoltre una funzione di ricerca che consente al docente di individuare rapidamente lezioni, compiti o valutazioni per data, studente o tipologia. Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli studenti dell'inserimento di nuovi compiti, della registrazione di un nuovo voto o della pubblicazione di una nuova lezione nel registro. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni salvate e una corretta gestione dei permessi tra docenti e studenti.



2. Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Il sistema consente l'accesso di docenti e studenti a una piattaforma digitale mediante autenticazione tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo (studente o docente), oltre a fornire nome, cognome ed indirizzo email istituzionale. Ogni docente ha la possibilità di creare e gestire una o più classi virtuali, ciascuna identificata da un nome e da un codice univoco. L'iscrizione di uno studente a una classe può avvenire in due modalità: il docente può iscrivere direttamente lo studente alla classe, oppure lo studente stesso può iscriversi autonomamente inserendo il codice univoco della classe fornito dal docente. All'interno di ogni classe, il docente può registrare le lezioni svolte. Per ciascuna lezione devono essere specificati almeno la data, l'argomento trattato ed eventualmente una breve descrizione aggiuntiva. Le lezioni devono essere visualizzabili in ordine cronologico all'interno del registro della classe. Il docente può inoltre assegnare compiti o attività da svolgere a casa. Ogni compito è definito da un titolo, una descrizione, una data di assegnazione e, opzionalmente, una data di scadenza. Gli studenti possono consultare l'elenco dei compiti assegnati direttamente all'interno del proprio profilo personale, con l'indicazione della classe e della data associata. Il sistema consente al docente di registrare le valutazioni ottenute dagli studenti relativamente a prove scritte e interrogazioni orali. Per ciascuna valutazione devono essere indicati almeno lo studente a cui si riferisce, la data, la tipologia della prova, il voto assegnato ed eventualmente una breve nota descrittiva. Le tipologie di prova considerate dal sistema sono almeno due: "prova scritta" e "interrogazione". Ogni studente può visualizzare, all'interno del proprio profilo personale, l'elenco dei voti (usato come sinonimo di Valutazione) ricevuti per ciascuna classe, con indicazione della data, della tipologia di prova e del valore ottenuto. Deve inoltre essere possibile consultare una media dei voti relativa alla singola classe, calcolata sulla base delle valutazioni registrate dal docente. Il docente, attraverso una interfaccia dedicata, può visualizzare il registro completo di ciascuna classe, comprendente l'elenco delle lezioni svolte, dei compiti assegnati e delle valutazioni registrate. Deve inoltre essere possibile accedere al dettaglio delle informazioni relative a ciascuno studente, visualizzando lo storico dei voti (usato come sinonimo di Valutazione) ricevuti e l'elenco dei compiti assegnati alla classe di appartenenza. Il sistema mantiene lo storico di tutte le attività svolte per ciascuna classe. Gli studenti possono consultare l'elenco delle lezioni già svolte, dei compiti assegnati e dei voti (usato come sinonimo di Valutazione) ricevuti. I docenti, attraverso un'interfaccia dedicata, possono monitorare l'andamento complessivo della classe, visualizzando il numero di lezioni registrate, il numero di compiti assegnati, la media generale della classe e il numero totale di valutazioni inserite in un determinato intervallo di tempo. Per semplicità, si può supporre che ogni classe sia gestita da un singolo docente e che ogni voto sia espresso in forma numerica. Il sistema include inoltre una funzione di ricerca che consente al docente di individuare rapidamente lezioni, compiti o valutazioni per data, studente o tipologia. Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche (classe: Notifica) che avvisi gli studenti dell'inserimento di nuovi compiti, della registrazione di un nuovo voto (usato come sinonimo di Valutazione) o della pubblicazione di una nuova lezione nel registro. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni salvate e una corretta gestione dei permessi tra docenti e studenti.

LEGENDA: Classe Attributo Funzionalità Attore Classe-Attore

2.2 Revisione dei requisiti

1. *Il sistema deve offrire al Docente e allo Studente una funzionalità per registrarsi. (RF01)*
2. *Il sistema deve offrire al Docente e allo Studente una funzionalità per accedere. (RF02)*
3. *Di ogni Utente si vuole memorizzare ruolo (Docente/Studente), nome, cognome e indirizzo email istituzionale (e password). (RD01)*
4. *Il sistema deve offrire al Docente una funzionalità per creare classi virtuali. (RF03)*
5. *Di ogni Classe virtuale si vuole memorizzare un nome e un codice univoco. (RD02)*
6. *Il sistema deve offrire al Docente una funzionalità per iscrivere lo studente alla classe virtuale. (RF04)*
7. *Il sistema deve offrire allo Studente una funzionalità per iscriversi autonomamente alla classe virtuale inserendo il codice univoco della classe. (RF05)*
8. *Il sistema deve offrire al Docente una funzionalità per registrare le lezioni svolte per una classe virtuale. (RF06)*
9. *Di ogni Lezione si vuole memorizzare data, argomento trattato ed eventualmente una breve descrizione aggiuntiva. (RD03)*
10. *Il sistema deve offrire allo Studente una funzionalità per visualizzare all'interno del registro della classe le lezioni in ordine cronologico. (RF07)*
11. *Il sistema deve offrire al Docente una funzionalità per assegnare compiti o attività da svolgere a casa. (RF08)*
12. *Di ogni Compito si vuole memorizzare titolo, descrizione, data di assegnazione e, opzionalmente, data di scadenza. (RD04)*
13. *Il sistema deve offrire allo Studente una funzionalità per consultare l'elenco dei compiti assegnati all'interno del proprio profilo personale specificando classe e data. (RF09)*
14. *Il sistema deve offrire al Docente una funzionalità per registrare le valutazioni ottenute dallo Studente in prove scritte e interrogazioni orali. (RF10)*
15. *Di ogni Valutazione si vuole memorizzare lo studente a cui si riferisce, la data, la tipologia della prova (prova scritta/interrogazione), il voto assegnato ed eventualmente una breve nota descrittiva. (RD05)*
16. *Il sistema deve offrire allo Studente una funzionalità per visualizzare, all'interno del proprio profilo personale, l'elenco dei voti ricevuti per ciascuna classe selezionata, con indicazione della data, della tipologia di prova e del valore ottenuto. (RF11)*
17. *Il sistema deve offrire allo Studente una funzionalità per consultare una media dei voti relativa alla singola classe. (RF12)*
18. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare la media generale di ciascuna classe in un determinato intervallo di tempo. (RF13)*
19. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare l'elenco delle lezioni svolte. (RF14)*
20. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare il numero delle lezioni svolte in un determinato intervallo di tempo. (RF15)*
21. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare l'elenco dei compiti assegnati. (RF16)*
22. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare il numero dei compiti assegnati in un determinato intervallo di tempo. (RF17)*
23. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare l'elenco delle valutazioni del singolo studente. (RF18)*
24. *Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare il numero delle valutazioni registrate in un determinato intervallo di tempo. (RF19)*
25. *Il sistema deve memorizzare lo storico di tutte le attività svolte per ciascuna classe. (RF20)*
26. *Il sistema deve prevedere che ogni classe sia gestita da un singolo docente. (RD06)*
27. *Il sistema deve prevedere i voti espressi in forma numerica. (RD07)*



28. *Il sistema deve offrire al Docente una funzionalità per ricercare rapidamente le lezioni svolte per data. (RF21)*
29. *Il sistema deve offrire al Docente una funzionalità per ricercare rapidamente i compiti assegnati per data di assegnazione o data di scadenza. (RF22)*
30. *Il sistema deve offrire al Docente una funzionalità per ricercare rapidamente le valutazioni per data, studente o tipologia (scritta/orale). (RF23)*
31. *Il sistema deve essere multipiattaforma: deve essere accessibile via web sia da desktop che da dispositivi mobili. (RQ-QUAL-01)*
32. *Il sistema deve avere un'interfaccia grafica intuitiva e responsiva. (RQ-QUAL-02)*
33. *Il sistema deve inviare una notifica agli studenti in seguito all'inserimento di nuovi compiti, alla registrazione di un nuovo voto o alla pubblicazione di una nuova lezione nel registro. (RF24+V01)*
34. *Il sistema deve garantire la protezione dei dati personali e la sicurezza delle informazioni salvate. (RQ-QUAL-03)*
35. *Il sistema deve garantire una corretta gestione dei permessi per docenti e studenti. (RQ-QUAL-04)*

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Utente	Una generica persona (docente, studente) che intende registrarsi per utilizzare il registro elettronico	-
Docente	Utente della piattaforma che crea e gestisce una o più classi, registra lezioni, assegna compiti, inserisce valutazioni e monitora l'andamento degli studenti.	-
Studente	Utente della piattaforma che può iscriversi alle classi, consultare lezioni e compiti, visualizzare le proprie valutazioni e la relativa media.	-
Classe virtuale	Ambiente virtuale creato e gestito da un docente nel quale vengono organizzati studenti, lezioni, compiti e valutazioni.	Classe
Lezione	Attività didattica svolta nella classe e registrata dal docente indicando almeno data e argomento ed eventualmente una descrizione aggiuntiva.	Attività
Registro	Insieme organizzato delle informazioni relative alle attività svolte in una classe, comprendente lezioni, compiti e valutazioni.	-
Compito	Attività assegnata dal docente agli studenti della classe, caratterizzata da titolo, descrizione, data di assegnazione ed eventuale data di scadenza.	Attività
Valutazione	Registrazione effettuata dal docente relativa al risultato ottenuto da uno studente in una prova. Comprende studente, data, tipologia della prova, voto (numerico) ed eventuale nota.	Voto (non sempre, talvolta è un attributo),
Sistema di notifiche	Sistema che genera e invia automaticamente avvisi agli studenti quando vengono registrate nuove attività nella classe, come la pubblicazione di una nuova lezione,	-

	l'assegnazione di un nuovo compito o l'inserimento di una nuova valutazione.	

2.4 Classificazione dei requisiti

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RF01	Il sistema deve offrire al Docente e allo Studente una funzionalità per registrarsi	1
RF02	Il sistema deve offrire al Docente e allo Studente una funzionalità per accedere.	2
RF03	Il sistema deve offrire al Docente una funzionalità per creare classi virtuali.	4
RF04	Il sistema deve offrire al Docente una funzionalità per iscrivere lo studente alla classe virtuale.	6
RF05	Il sistema deve offrire allo Studente una funzionalità per iscriversi autonomamente alla classe virtuale inserendo il codice univoco della classe.	7
RF06	Il sistema deve offrire al Docente una funzionalità per registrare le lezioni svolte per una classe virtuale.	8
RF07	Il sistema deve offrire allo Studente una funzionalità per visualizzare all'interno del registro della classe le lezioni in ordine cronologico.	10
RF08	Il sistema deve offrire al Docente una funzionalità per assegnare compiti o attività da svolgere a casa.	11
RF09	Il sistema deve offrire allo Studente una funzionalità per consultare l'elenco dei compiti assegnati all'interno del proprio profilo personale specificando classe e data.	13
RF10	Il sistema deve offrire al Docente una funzionalità per registrare le valutazioni ottenute dallo Studente in prove scritte e interrogazioni orali.	14
RF11	Il sistema deve offrire allo Studente una funzionalità per visualizzare, all'interno del proprio profilo personale, l'elenco dei voti ricevuti per ciascuna classe selezionata, con indicazione della data, della tipologia di prova e del valore ottenuto	16
RF12	Il sistema deve offrire allo Studente una funzionalità per consultare una media dei voti relativa alla singola classe.	17
RF13	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare la media generale di ciascuna classe in un determinato intervallo di tempo.	18
RF14	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare l'elenco delle lezioni svolte.	19

RF15	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare il numero delle lezioni svolte in un determinato intervallo di tempo.	20
RF16	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare l'elenco dei compiti assegnati.	21
RF17	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare il numero dei compiti assegnati in un determinato intervallo di tempo.	22
RF18	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare l'elenco delle valutazioni del singolo studente.	23
RF19	Il sistema deve offrire al Docente un'interfaccia dedicata per visualizzare il numero delle valutazioni registrate in un determinato intervallo di tempo.	24
RF20	Il sistema deve memorizzare lo storico di tutte le attività svolte per ciascuna classe	25
RF21	Il sistema deve offrire al Docente una funzionalità per ricercare rapidamente le lezioni svolte per data.	28
RF22	Il sistema deve offrire al Docente una funzionalità per ricercare rapidamente i compiti assegnati per data di assegnazione o data di scadenza.	29
RF23	Il sistema deve offrire al Docente una funzionalità per ricercare rapidamente le valutazioni per data, studente o tipologia (scritta/orale).	30
RF24	Il sistema deve inviare una notifica agli studenti in seguito all'inserimento di nuovi compiti, alla registrazione di un nuovo voto o alla pubblicazione di una nuova lezione nel registro	33

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RD01	Di ogni Utente si vuole memorizzare ruolo (Docente/Studente), nome, cognome e indirizzo email istituzionale (e password).	3
RD02	Di ogni Classe virtuale si vuole memorizzare un nome e un codice univoco.	5
RD03	Di ogni Lezione si vuole memorizzare data, argomento trattato ed eventualmente una breve descrizione aggiuntiva.	9
RD04	Di ogni Compito si vuole memorizzare titolo, descrizione, data di assegnazione e, opzionalmente, data di scadenza.	12
RD05	Di ogni Valutazione si vuole memorizzare lo studente a cui si riferisce, la data, la tipologia della prova (prova scritta/interrogazione), il voto assegnato ed eventualmente una breve nota descrittiva.	15

RD06	Il sistema deve prevedere che ogni classe sia gestita da un singolo docente.	26
RD07	Il sistema deve prevedere i voti espressi in forma numerica.	27

2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010)

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUAL-01	Portability (Adaptability)	Il sistema deve essere multipiattaforma: deve essere accessibile via web sia da desktop che da dispositivi mobili.	1) Percentuale di ambienti desktop e mobili supportati in cui il sistema risulta correttamente accessibile e utilizzabile tramite browser web	1)100% degli ambienti dichiarati supportati	1)Test di compatibilità su diversi browser e dispositivi desktop e mobili, verificando il corretto accesso al sistema e il funzionamento delle principali funzionalità
RQ-QUAL-02	Usabilità (operability, learnability)	Il sistema deve avere un'interfaccia grafica intuitiva e responsiva.	1)Percentuale di attività completate correttamente dagli utenti senza assistenza 2) Percentuale di risoluzioni testate in cui gli utenti riescono a utilizzare correttamente le funzionalità principali dell'interfaccia	1) ≥ 90,0% 2)100,0%	1)Test di usabilità con utenti rappresentativi 2) Test di usabilità a differenti risoluzioni, verificando la leggibilità e la facilità di interazione con gli elementi grafici
RQ-QUAL-03	Sicurezza (confidentiality, integrity, authenticity)	Il sistema deve garantire la protezione dei dati personali e la sicurezza delle informazioni salvate.	1)Percentuale di dati personali protetti mediante cifratura	1)100% 2)100% 3)100%	1) Test di sicurezza per verificare che i dati personali siano protetti mediante cifratura durante

			2) Percentuale di tentativi di alterazione non autorizzata dei dati salvati correttamente impediti o rilevati 3) Percentuale di tentativi di autenticazione con credenziali non valide correttamente respinti.		la memorizzazione 2) Test di integrità mediante simulazione di alterazioni dei dati e verifica del loro rilevamento 3) Test di autenticazione mediante credenziali errate, inesistenti o scadute, verificando che l'accesso venga negato
RQ-QUAL-04	Sicurezza (confidentiality, integrity)	Il sistema deve garantire una corretta gestione dei permessi per docenti e studenti.	1) Percentuale di accessi alle risorse gestiti correttamente in base ai permessi assegnati 2) Percentuale di operazioni di modifica ed eliminazione gestite correttamente in base ai permessi assegnati	1)100% 2)100%	1) Test di autorizzazione per verificare che docenti e studenti possano accedere esclusivamente alle risorse consentite 2) Test di autorizzazione per verificare che le operazioni consentite siano eseguite e quelle non autorizzate siano bloccate

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
Vo1	Per l'invio delle notifiche agli studenti il sistema deve appoggiarsi ad un sistema esterno.	33
Vo2	Il voto di una Valutazione deve essere compreso tra 1 e 10 (estremi inclusi) e deve essere multiplo di 0.25.	-
Vo3	La data di una Valutazione non deve essere successiva a quella odierna e il giorno non deve essere domenica.	-
Vo4	La nota descrittiva di una Valutazione deve avere al più 200 caratteri ed è opzionale.	-
Vo5	Il nome di una classe virtuale è obbligatorio e deve avere al più 50 caratteri.	-
Vo6	Le classi virtuali di uno stesso docente non possono avere lo stesso nome.	-
Vo7	Il codice di una classe virtuale deve avere esattamente 10 caratteri, di cui i primi 4 sono "CLS-".	-
Vo8	Le classi virtuali del sistema devono avere codici diversi.	-
Vo9	Uno studente non può iscriversi più volte alla stessa classe virtuale.	-

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Utente
- Studente
- Docente

Attori Secondari:

- Sistema notifiche

Casi d'uso:

- UC1: Registrazione
- UC2: Visualizzazione attività classe
- UC3: Visualizza compiti assegnati
- UC4: Visualizza valutazioni e media per classe
- UC5: Visualizza cronologia lezioni della classe
- UC6: Iscrizione studente ad una classe
- UC7: Iscrizione tramite codice classe
- UC8: Iscrizione diretta dello studente
- UC9: Visualizza andamento complessivo classe
- UC10: Visualizzazione registro classe
- UC11: Visualizza valutazioni singolo studente
- UC12: Visualizza lezioni registrate per la classe virtuale
- UC13: Visualizza compiti assegnati per la classe virtuale
- UC14: Ricerca Lezioni, Compiti, Valutazioni
- UC15: Creazione classe virtuale
- UC16: Registrazione voce di registro
- UC17: Registra lezione svolta
- UC18: Registra valutazione
- UC19: Registra compito

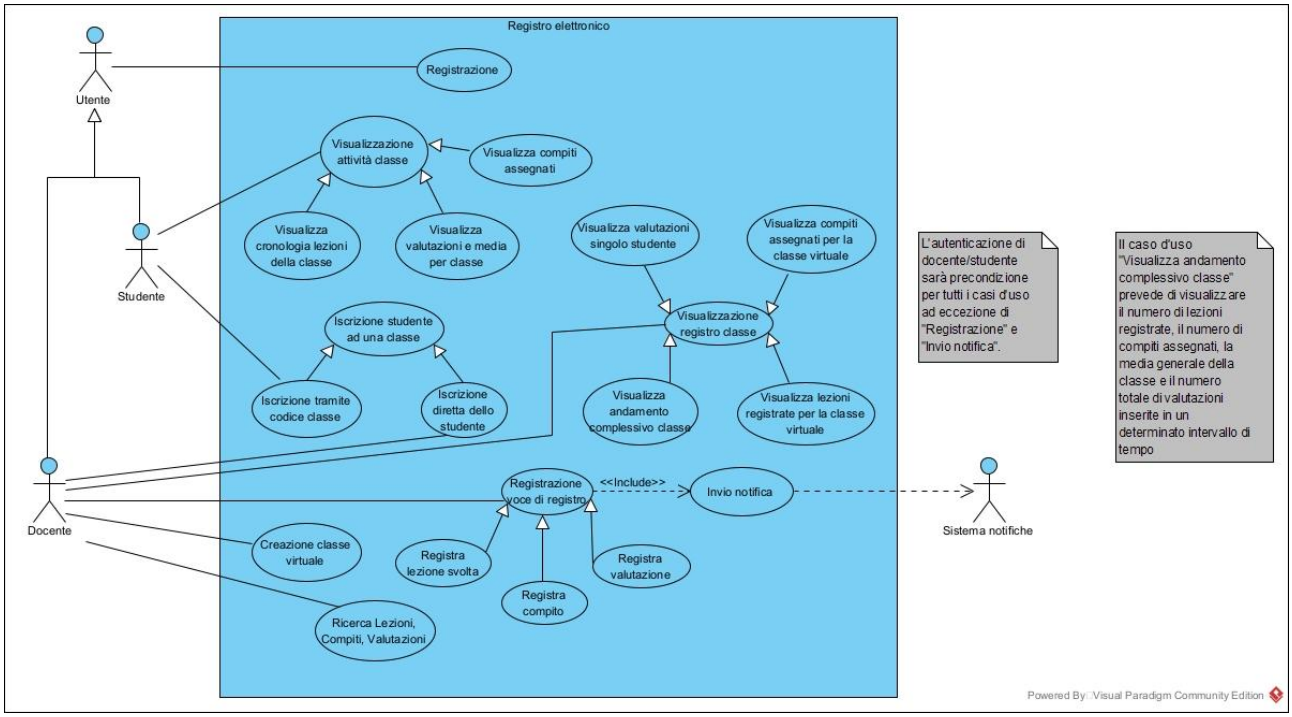
Casi d'uso di inclusione:

- UC20: Invio notifica

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1: Registrazione	Utente		-	RF01
UC2: Visualizzazione attività classe	Studente	-	-	RF09, RF11, RF12, RF07
UC3: Visualizza compiti assegnati	Studente	-	Extends Visualizzazione attività classe	RF09
UC4: Visualizza valutazioni e media per classe	Studente	-	Extends Visualizzazione attività classe	RF11, RF12
UC5: Visualizzazione cronologia lezioni della classe	Studente	-	Extends Visualizzazione attività classe	RF07
UC6: Iscrizione studente ad una classe	-	-	-	RF04, RF05
UC7: Iscrizione tramite codice classe	Studente	-	Extends Iscrizione studente ad una classe	RF05

UC8: Iscrizione diretta dello studente	Docente	-	Extends Iscrizione studente ad una classe	RF04
UC9: Visualizza andamento complessivo classe	Docente	-	Extends Visualizzazione registro classe	RF13, RF15, RF17, RF19
UC10: Visualizzazione registro classe	Docente	-	-	RF13, RF15, RF17, RF19, RF18, RF14, RF16
UC11: Visualizza valutazioni singolo studente	Docente	-	Extends Visualizzazione registro classe	RF18
UC12: Visualizza lezioni registrate per la classe virtuale	Docente	-	Extends Visualizzazione registro classe	RF14
UC13: Visualizza compiti assegnati per la classe virtuale	Docente	-	Extends Visualizzazione registro classe	RF16
UC14: Ricerca Lezioni, Compiti, Valutazioni	Docente	-	-	RF21, RF22, RF23
UC15: Creazione classe virtuale	Docente	-	-	RF03
UC16: Registrazione voce di registro	Docente	-	Include Invio notifica	RF06, RF08, RF10
UC17: Registra lezione svolta	Docente	-	Extends Registrazione voce di registro	RF06
UC18: Registra valutazione	Docente	-	Extends Registrazione voce di registro	RF10
UC19: Registra compito	Docente	-	Extends Registrazione voce di registro	RF08
UC20: Invio notifica	-	Sistema notifiche	Incluso in Registrazione voce di registro	RF24

2.5.2 Diagramma dei casi d'uso



2.5.3 Scenari

Caso d'uso:	**Creazione classe virtuale**
Attore primario	Docente
Attore secondario	-
Descrizione	Il Docente crea una classe virtuale specificandone il nome.
Pre-Condizioni	Il Docente ha effettuato l'accesso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente, entrato nell'Area Docente del registro elettronico, clicca il pulsante per creare una nuova classe virtuale. 2. Il Sistema mostra un form per permettere l'inserimento del nome della classe. 3. Il Docente compila il form e preme il pulsante per confermare la creazione. 4. **if** nome non inserito 4.1. Il Sistema vieta l'operazione e comunica al Docente che è necessario inserire il nome della classe. **5. else if** nome con lunghezza maggiore di 50 caratteri 5.1. Il Sistema vieta l'operazione e comunica al Docente che il nome della classe può contenere al massimo 50 caratteri. 6. **else if** nome già utilizzato da un'altra classe dello stesso Docente 6.1. Il Sistema vieta l'operazione e comunica al Docente che possiede già una classe con il nome inserito. 7. **else** 7.1. Il Sistema assegna un codice univoco alla classe virtuale. 7.2. Il Sistema memorizza la classe virtuale in maniera persistente. 7.3. La nuova classe figura, con il nome inserito e il codice univoco generato, nell'elenco delle classi del Docente mostrato nell'Area Docente.
Post-Condizioni	La classe virtuale è stata creata e resa persistente nel sistema e il codice univoco per essa generata non è più disponibile per altre classi.

Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3 se il Docente preme il pulsante per annullare l'operazione, il caso d'uso è interrotto.

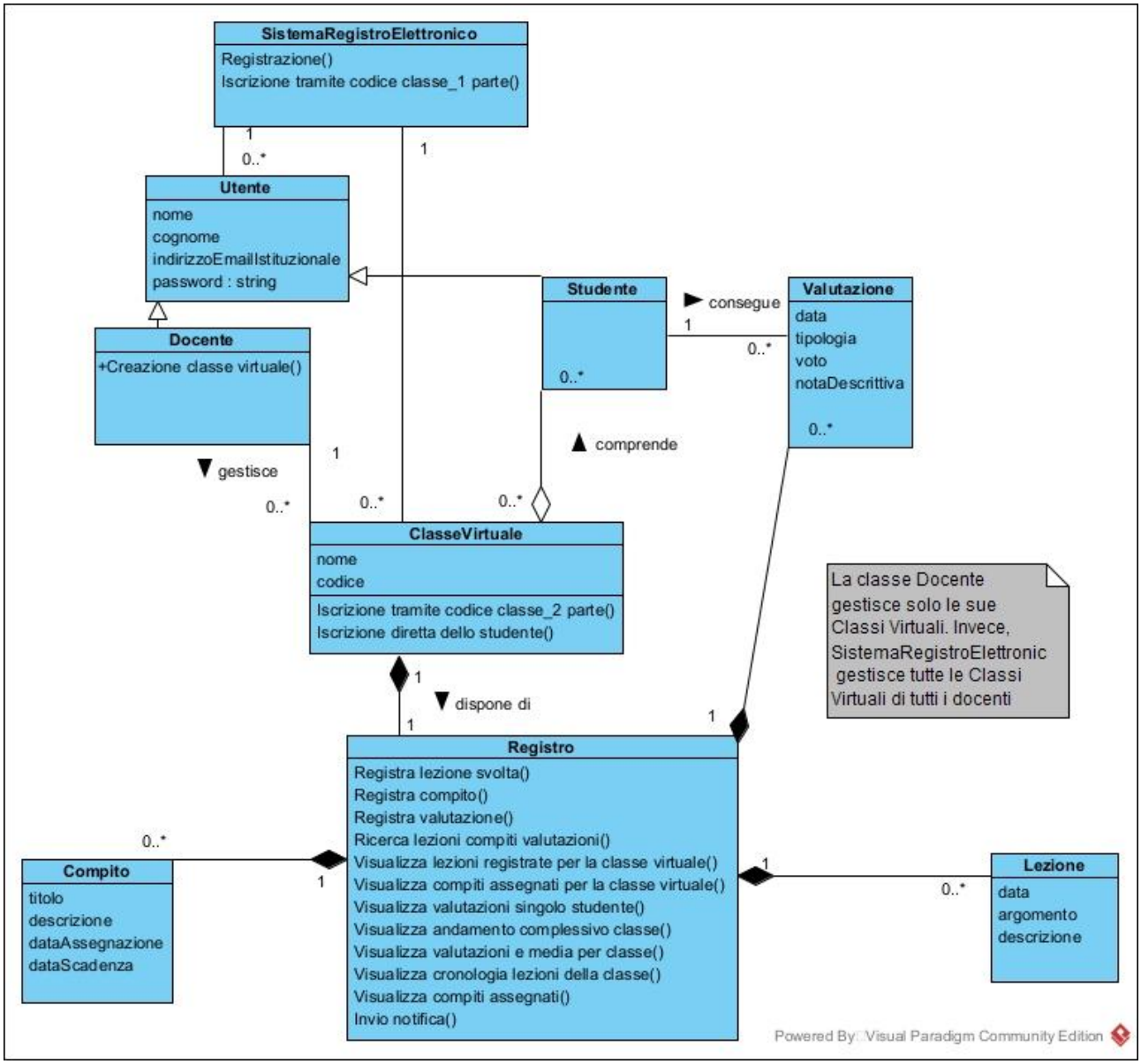
Caso d'uso:	**Iscrizione tramite codice classe**
Attore primario	Studente
Attore secondario	-
Descrizione	Lo Studente si iscrive ad una classe virtuale attraverso il codice univoco ad essa associato.
Pre-Condizioni	Lo Studente ha effettuato l'accesso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente, entrato nell'Area Studente del registro elettronico, clicca il pulsante per iscriversi ad una classe virtuale. 2. Il sistema mostra un form per permettere l'inserimento del codice della classe. 3. Lo Studente compila il form e preme il pulsante per confermare l'iscrizione. 4. **if** codice non inserito 4.1. Il Sistema vieta l'operazione e comunica allo Studente che è necessario inserire il codice 5. **else if** codice con lunghezza diversa da 10 5.1. Il Sistema vieta l'operazione e comunica allo Studente che il codice deve essere composto da 10 caratteri. 6. **else if** codice non inizia con "CLS-" 6.1. Il Sistema vieta l'operazione e comunica allo Studente che il codice deve iniziare con il prefisso "CLS-". 7. **else if** codice corrispondente a una classe del sistema 7.1. **if** Studente non già iscritto 7.1.1. Il Sistema memorizza l'iscrizione dello Studente alla Classe virtuale. 7.1.2. La nuova classe a cui lo Studente si è iscritto figurerà con nome e codice nell'elenco delle classi di cui lo Studente fa parte che è mostrato a schermo nell'area studente. 7.2. **else** 7.2.1. Il Sistema vieta l'operazione e comunica allo Studente che è già iscritto a quella classe. **8. else** 8.1 Il Sistema vieta l'operazione e comunica allo Studente che il codice è inesistente.
Post-Condizioni	Lo Studente risulta registrato nella classe virtuale.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3 se lo Studente preme il pulsante per annullare l'operazione, il caso d'uso è interrotto.

Caso d'uso:	Registra valutazione
Attore primario	Docente
Attore secondario	-
Descrizione	Il Docente registra una valutazione relativa a uno Studente per una sua classe.
Pre-Condizioni	Il Docente ha effettuato l'accesso. La classe virtuale ha almeno uno Studente registrato.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente entra nella schermata relativa ad una sua classe virtuale. 2. Il Docente apre il registro della classe. 3. Il Sistema mostra al docente tutte le azioni che può effettuare con il registro della classe. 4. Il Docente seleziona l'opzione per registrare una valutazione nel registro. 5. Il Sistema mostra una schermata per la registrazione di una nuova valutazione relativa alla classe selezionata. 6. Il Docente seleziona lo studente (da selezionare tra tutti gli studenti iscritti alla classe), la tipologia (da selezionare tra prova scritta e interrogazione), la data dal calendario, il voto (da selezionare tra i voti ammessi) e opzionalmente inserisce una breve nota descrittiva. 7. Il Docente compila il form e clicca sul pulsante per registrare la valutazione. 8. **if** tipologia non selezionata 8.1. Il Sistema vieta l'operazione e comunica al Docente che è necessario selezionare la tipologia della valutazione. 9. **else if** voto non selezionato 9.1. Il Sistema vieta l'operazione e comunica al Docente che è necessario selezionare un voto. 10. **else if** Studente non selezionato 10.1 Il Sistema vieta l'operazione e comunica al Docente che è necessario selezionare uno Studente. 11. **else if** data non selezionata 11.1. Il Sistema vieta l'operazione e comunica al Docente che è necessario selezionare una data. 12. **else if** data selezionata successiva alla data corrente 12.1. Il Sistema vieta l'operazione e comunica al Docente che la data della valutazione non può essere successiva alla data corrente. 13. **else if** data selezionata corrisponde a una domenica 13.1. Il Sistema vieta l'operazione e comunica al Docente che la data della valutazione non può corrispondere a una domenica. 14. **else if** nota descrittiva contiene più di 200 caratteri 14.1. Il Sistema vieta l'operazione e comunica al Docente che la nota può contenere al massimo 200 caratteri. 15. **else** 15.1. Il Sistema registra la valutazione in maniera persistente. 15.2. Il Sistema comunica al Docente l'avvenuta registrazione della valutazione. 15.3. <<include>> Invio notifica.
Post-Condizioni	La valutazione dello studente per quella classe virtuale risulta registrata in maniera persistente.
Casi d'uso correlati	*Invio notifica*

Sequenza di eventi alternativi	

Caso d'uso:	**Visualizza valutazioni singolo studente**
Attore primario	Docente
Attore secondario	-
Descrizione	Il Docente seleziona uno Studente e ne visualizza le valutazioni per la classe virtuale.
Pre-Condizioni	Il Docente ha effettuato l'accesso. La classe virtuale ha almeno uno Studente registrato.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente entra nella schermata relativa ad una sua classe virtuale. 2. Il Docente apre il registro della classe. 3. Il Sistema mostra al docente tutte le azioni che può effettuare con il registro della classe. 4. Il Docente seleziona l'opzione per visualizzare le valutazioni degli studenti 5. Il Sistema mostra al Docente una schermata dedicata alla visualizzazione delle valutazioni degli studenti della classe virtuale selezionata all'inizio del caso d'uso dal Docente 6. Il Docente seleziona tra tutti gli studenti della classe lo Studente di cui vuole visualizzare le valutazioni 7. Il Sistema mostra al Docente le valutazioni di quello Studente
Post-Condizioni	-
Casi d'uso correlati	-
Sequenza di eventi alternativi	-

2.6 Diagramma delle classi



A seguire, una tabella che riassume alcune responsabilità:

RESPONSABILITÀ	CLASSE
Registrazione	**SistemaRegistroElettronico**
Creazione classe virtuale	**Docente**
Registra lezione svolta	**Registro**
Registra Compito	**Registro**
Registra valutazione	**Registro**
Ricerca Lezioni, Compiti, Valutazioni	**Registro**
Visualizza lezioni registrate per la classe virtuale	**Registro**
Visualizza compiti assegnati per la classe virtuale	**Registro**

Visualizza valutazioni singolo studente	**Registro**
Visualizzazione andamento complessivo classe	**Registro**
Visualizza valutazioni e media per classe	**Registro**
Visualizza cronologia lezioni della classe	**Registro**
Visualizza compiti assegnati	**Registro**
Invio notifica	**Registro**
Iscrizione tramite codice classe	**SistemaRegistroElettronico e poi ClasseVirtuale**
Iscrizione diretta dello studente	**ClasseVirtuale**

Commento scelte principali Domain Model:
Abbiamo scelto di distinguere la classe "Registro" dalla classe "ClasseVirtuale", in modo tale da separare le responsabilità delle due classi, facendo in modo che ClasseVirtuale si occupi sostanzialmente della gestione anagrafica e informativa della classe, mantenendo l'associazione con il docente che la crea e un contenimento lasco con gli studenti iscritti (lasco perché il ciclo di vita dello studente è indipendente da quello di una data classe virtuale ovviamente). Invece Registro la utilizziamo per tutte quelle responsabilità operative, legate alle attività didattiche come compiti, valutazioni e lezioni svolte. In particolare, ClasseVirtuale è in una relazione di contenimento stretto con Registro in virtù della dipendenza del ciclo di vita di Registro da quello di ClasseVirtuale: quando viene creata una ClasseVirtuale è creato il Registro correlato e quando è eliminata viene cancellato anche il relativo Registro con tutti i compiti, le valutazioni e le lezioni registrate.
Registro contiene strettamente Valutazione, Compito e Lezione perché i loro cicli di vita sono dipendenti dal ciclo di vita del Registro: quando è creato il Registro sono automaticamente create delle strutture dati (su cui non ci pronunciamo in fase di analisi), inizialmente vuote, relative ai compiti, alle valutazioni e alle lezioni per quella specifica ClasseVirtuale; allo stesso modo, quando eliminiamo fisicamente il registro cancelliamo anche tutte le valutazioni, i compiti e le lezioni così che non ci siano oggetti orfani.
"È previsto un sistema di notifiche che avvisi gli studenti dell'inserimento di nuovi compiti, della registrazione di un nuovo voto o della pubblicazione di una nuova lezione nel registro". Questo è un aspetto che valuteremo meglio nelle fasi successive a questa, per il momento ci limitiamo ad assegnare la responsabilità di invio della notifica alla classe Registro come spiegheremo nel paragrafo che segue.

Commento decisioni sulle responsabilità:

Registrazione- SistemaRegistroElettronico
Applicando il pattern GRASP Creator, abbiamo verificato che nel Domain Model non esistono classi del mondo reale che abbiano i giusti requisiti per creare istanze di Docente o Studente. Per risolvere questo problema, abbiamo introdotto la classe SistemaRegistroElettronico applicando il pattern Pure Fabrication, che si occuperà tra le varie cose della registrazione utente. (PATTERN GRASP: PURE FABRICATION)

Creazione classe virtuale (include istanziazione oggetto Classe Virtuale) -Docente

Abbiamo scelto Docente perché ha un'associazione con ClasseVirtuale e quindi manterrà una collezione che rappresenta le classi virtuali che gestisce; potrà perciò istanziare un nuovo oggetto ClasseVirtuale da aggiungere alla collezione ogni qualvolta deve essere creata una nuova ClasseVirtuale (PATTERN GRASP: CREATOR).

Registra lezione svolta (include istanziazione oggetto Lezione) -Registro
Registra Compito (include istanziazione oggetto Compito) -Registro
Registra valutazione (include istanziazione oggetto Valutazione) -Registro
Abbiamo scelto Registro per istanziare gli oggetti di queste 3 classi perché contiene strettamente tali classi e quindi mantiene una collezione per ciascuna di esse. Inoltre, la relazione di composizione impone dei vincoli in termini di ciclo di vita per cui l'oggetto contenitore (Registro) quando viene creato deve istanziare gli oggetti contenuti (Lezione, Valutazione, Compito). (PATTERN GRASP: CREATOR)

Ricerca Lezioni, Compiti, Valutazioni- Registro
Visualizza lezioni registrate per la classe virtuale - Registro
Visualizza compiti assegnati per la classe virtuale- Registro
Visualizza valutazioni singolo studente- Registro
Visualizzazione andamento complessivo classe - Registro
Abbiamo scelto Registro perché con una relazione di composizione mantiene l'elenco delle lezioni, dei compiti assegnati e delle valutazioni degli studenti iscritti alla classe e quindi può visualizzarne le informazioni. (PATTERN GRASP: INFORMATION EXPERT)

Visualizza valutazioni e media per classe (la richiede lo studente) – Registro
Lo studente vuole vedere le valutazioni e la media per una data classe. La responsabilità è di registro perché lo studente deve prima accedere alla classe specifica e quindi al registro associato e solo in quel momento potrà chiamare un metodo su registro che possiede le valutazioni per quella classe richiedendogli di visualizzare tutte le sue valutazioni. Prevediamo infatti come caso d'uso che lo studente possa vedere le valutazioni classe per classe, aprendone i registri corrispondenti, e non di tutte le classi assieme. (PATTERN GRASP: INFORMATION EXPERT)

Visualizza cronologia lezioni della classe - Registro
Visualizza compiti assegnati- Registro
Abbiamo scelto la classe Registro per queste responsabilità perché mantiene delle collezioni di compiti e lezioni in virtù dei contenimenti stretti con le due classi. (PATTERN GRASP: INFORMATION EXPERT)

Invio notifica- Registro
Abbiamo scelto la classe Registro perché la notifica viene inviata in seguito all'inserimento di un nuovo compito, alla registrazione di un nuovo voto o alla pubblicazione di una nuova lezione nel registro ed è Registro che si occupa di queste responsabilità. Per cui in seguito ad uno di questi eventi, Registro innesca anche l'invio della notifica.

Iscrizione tramite codice classe- SistemaRegistroElettronico e poi ClasseVirtuale
Quando l'attore studente vuole iscriversi ad una classe virtuale inserendone il codice la classe SistemaRegistroElettronico che mantiene una collezione delle classi virtuali del sistema effettua una ricerca della classe con quel codice e poi su di essa verrà chiamato il metodo per iscrivere lo studente alla classe. (PATTERN GRASP: INFORMATION EXPERT)

Iscrizione diretta dello studente- ClasseVirtuale

Abbiamo scelto ClasseVirtuale perché mantiene la collezione degli studenti iscritti. Di conseguenza, lei è l'unica "esperta" che ha il diritto di modificare quella collezione aggiungendo un nuovo membro. (PATTERN GRASP: INFORMATION EXPERT)

In generale nell'assegnazione delle responsabilità abbiamo tenuto in considerazione anche il PATTERN GRASP "LOW COUPLING" cercando di attribuire alle classi responsabilità che le richiedono di interagire soprattutto con le sue classi vicine



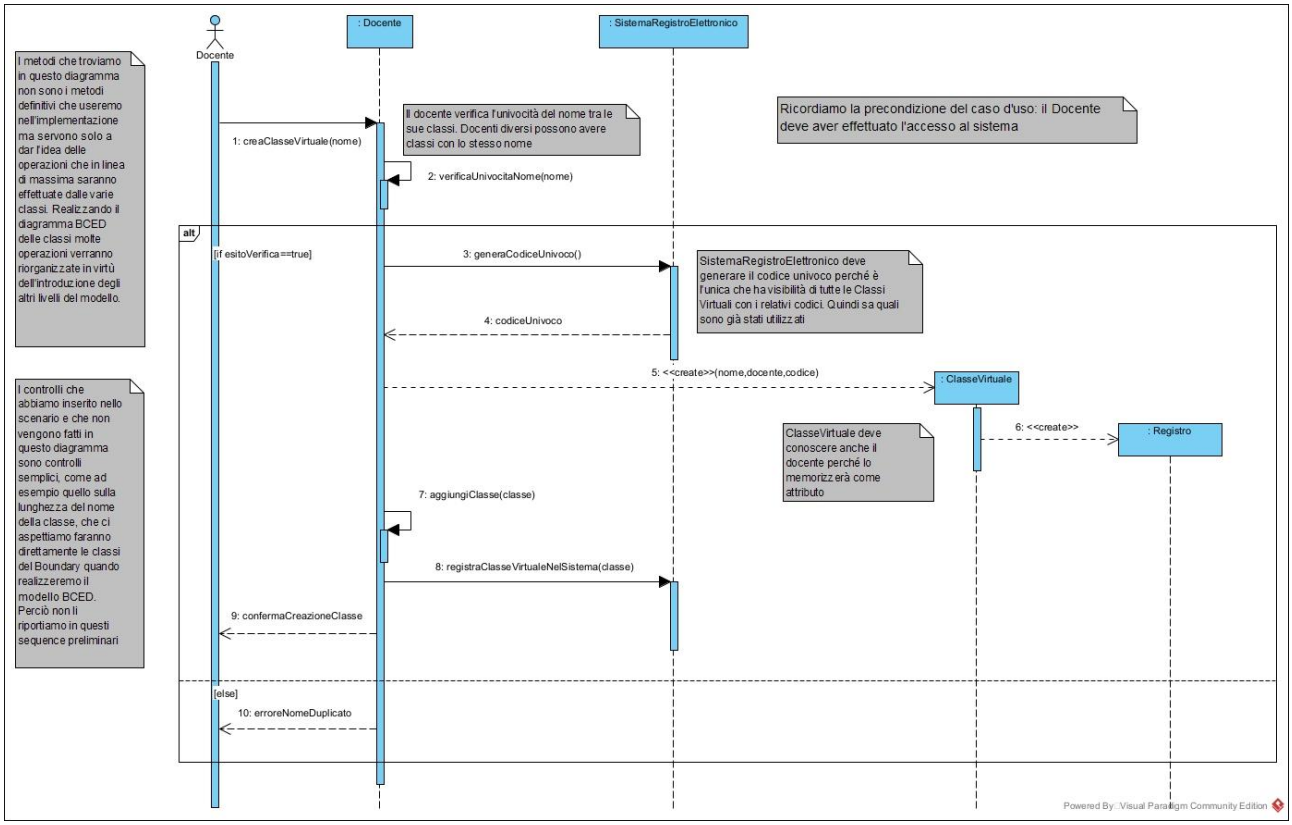
2.7 Diagrammi di sequenza

Si riportano i diagrammi di sequenza di analisi per i casi d'uso precedentemente individuati.

2.7.1 Creazione classe virtuale

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *"Creazione classe virtuale"*, ha fatto sorgere la necessità di definire alcuni metodi:

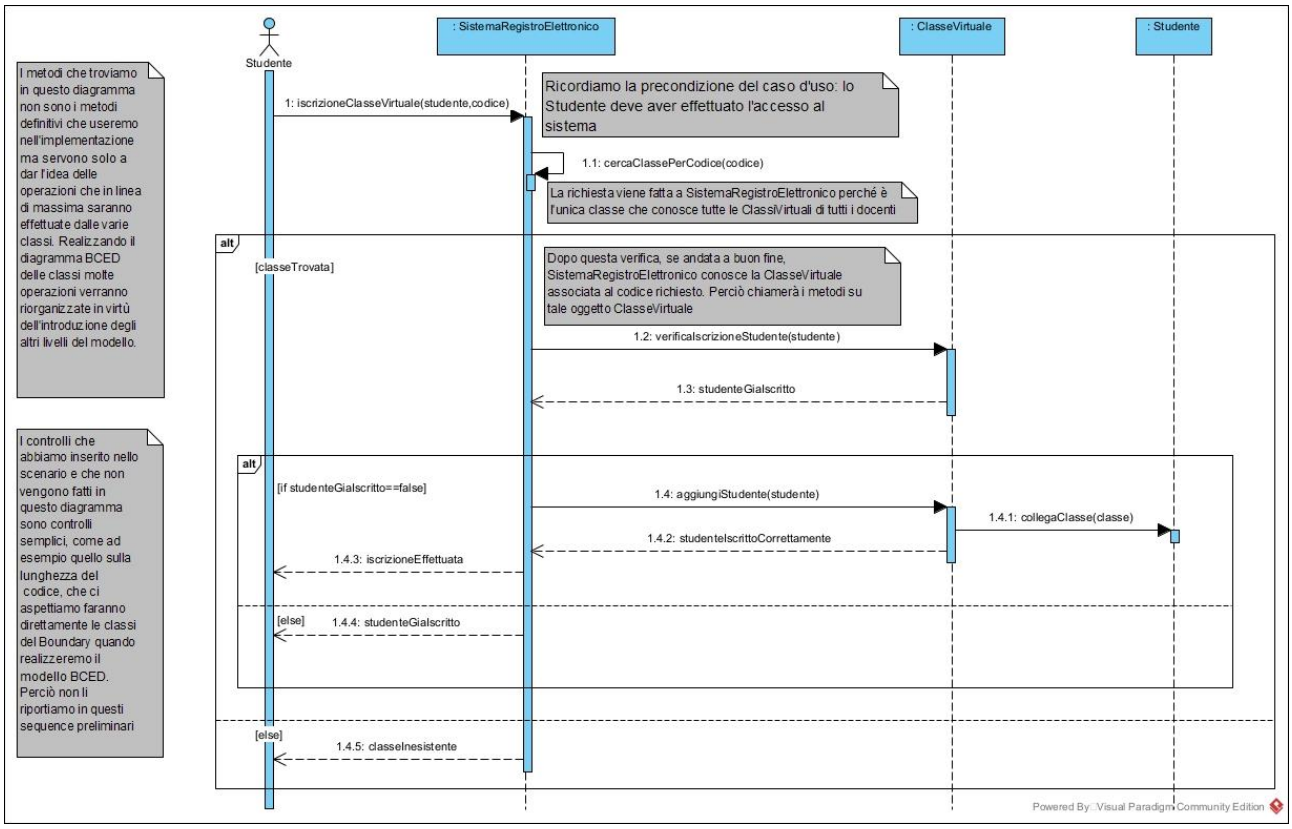
- Il metodo creaClasseVirtuale(nome) della classe Docente che coordina la creazione di una nuova classe virtuale prendendo il nome inserito dall'attore Docente.
- Il metodo verificaUnivocitaNome(nome) della classe Docente è utilizzato per controllare che il docente non possieda già una classe con lo stesso nome.
- Il metodo generaCodiceUnivoco() della classe SistemaRegistroElettronico che genera un codice non ancora associato ad altre classi virtuali.
- Il metodo aggiungiClasse(classe) della classe Docente è responsabile dell'inserimento della nuova classe nella collezione delle classi virtuali gestite dal docente.
- Il metodo registraClasseVirtualeNelSistema(classe) della classe SistemaRegistroElettronico registra la nuova classe nella collezione complessiva delle classi gestite dal sistema.



2.7.2 Iscrizione tramite codice classe

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *"Iscrizione tramite codice classe"*, ha fatto sorgere la necessità di definire alcuni metodi:

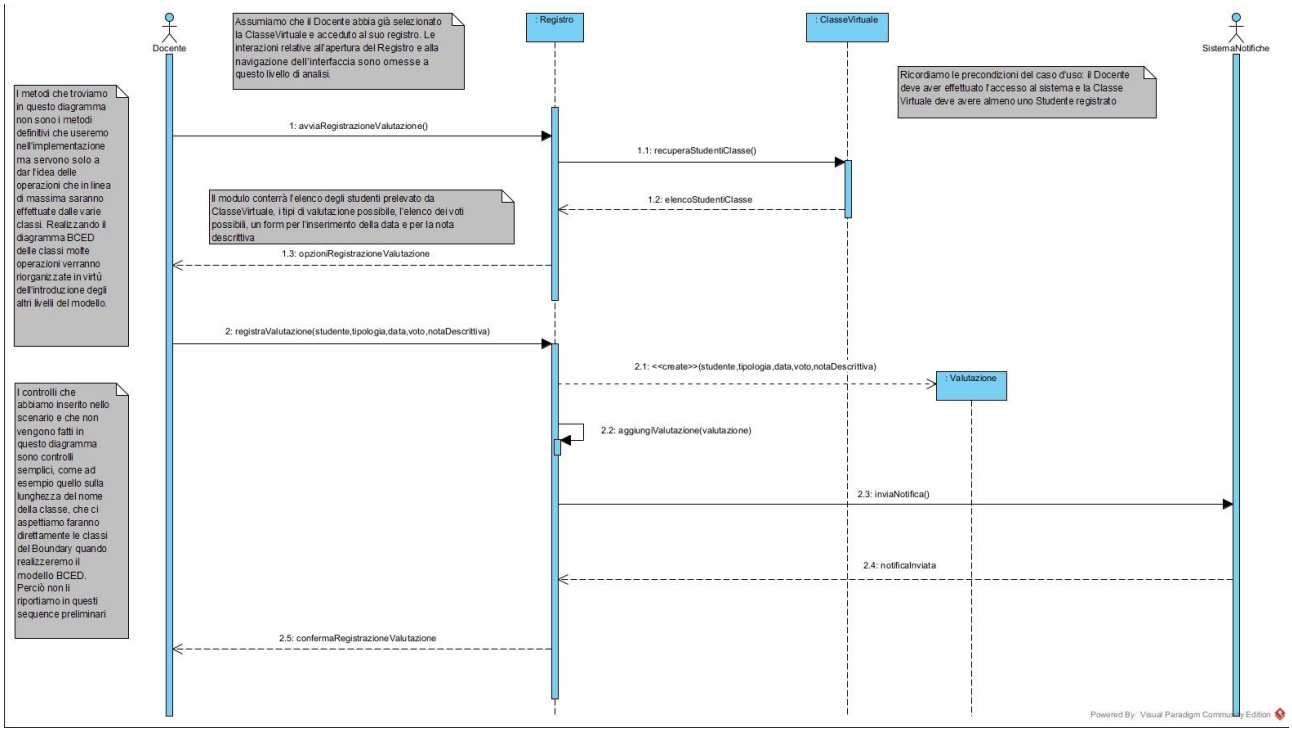
- Il metodo iscrizioneClasseVirtuale(studente,codice) della classe SistemaRegistroElettronico dirige lo svolgimento del processo di iscrizione.
- Il metodo cercaClassePerCodice(codice) della classe SistemaRegistroElettronico permette di cercare la classe virtuale, tra tutte quelle registrate sul sistema, associata al codice inserito. Restituisce la ClasseVirtuale trovata, se esiste, oppure un'indicazione di classe non trovata.
- Il metodo verificaIscrizioneStudente(studente) della classe Classe Virtuale controlla e restituisce un'indicazione relativa alla presenza dello studente nella collection degli studenti iscritti alla classe.
- Il metodo aggiungiStudente(studente) della classe Classe Virtuale è responsabile dell'inserimento dello studente nella collection degli studenti iscritti alla classe virtuale qualora non sia ancora iscritto.
- Il metodo collegaClasse(classe) della classe Studente permette di aggiungere alla collection delle classi frequentate dallo studente la nuova Classe Virtuale a cui lo studente si è iscritto. Questa operazione consente di mantenere coerenti entrambi i lati dell'associazione tra ClasseVirtuale e Studente: lo Studente mantiene la collection delle classi di cui fa parte e la ClasseVirtuale mantiene la collection degli studenti che la compongono.



2.7.3 Registra valutazione

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *"Registra valutazione"*, ha fatto sorgere la necessità di definire alcuni metodi:

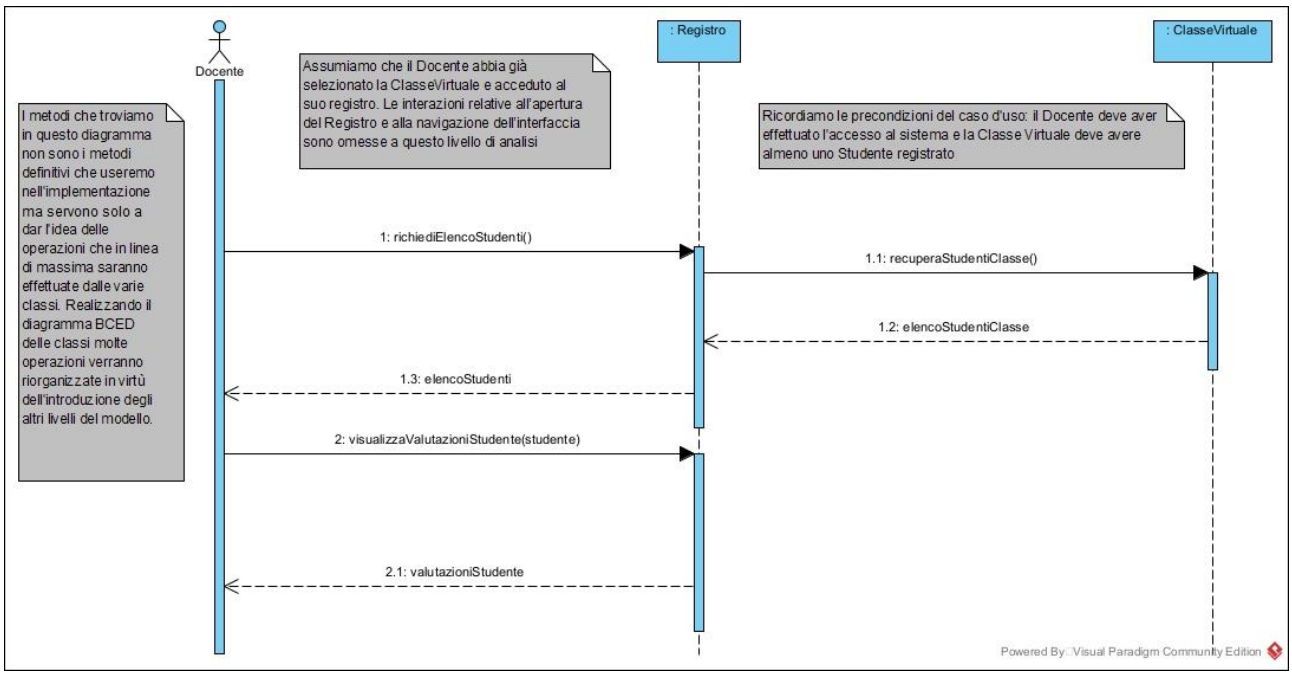
- Il metodo avviaRegistrazioneValutazione() della classe Registro gestisce la richiesta iniziale del docente per predisporre le informazioni da mostrare al Docente per registrare una nuova valutazione.
- Il metodo recuperaStudentiClasse() della classe Classe Virtuale (lo stesso del caso d'uso "Visualizza valutazioni singolo studente") accede alla collezione degli studenti iscritti e la restituisce.
- Il metodo registraValutazione(studente, tipologia, data, voto, notaDescrittiva) della classe Registro coordina la creazione e la memorizzazione della valutazione e il successivo invio della notifica.
- Il metodo aggiungiValutazione(valutazione) della classe Registro è utilizzato per l'inserimento della valutazione nella collezione delle valutazioni del registro.
- inviaNotifica() a questo livello d'analisi lo rappresentiamo come un messaggio rivolto all'attore esterno SistemaNotifiche. Sicuramente la classe Registro, avendo la responsabilità dell'invio della notifica, avrà un metodo inviaNotifica() che rappresenteremo nel diagramma delle classi raffinato che produrremo sulla base di questi sequence diagram. Sarà dettagliato meglio dopo aver costruito il modello BCED e chiarito meglio il flusso.



2.7.4 Visualizza valutazioni singolo studente

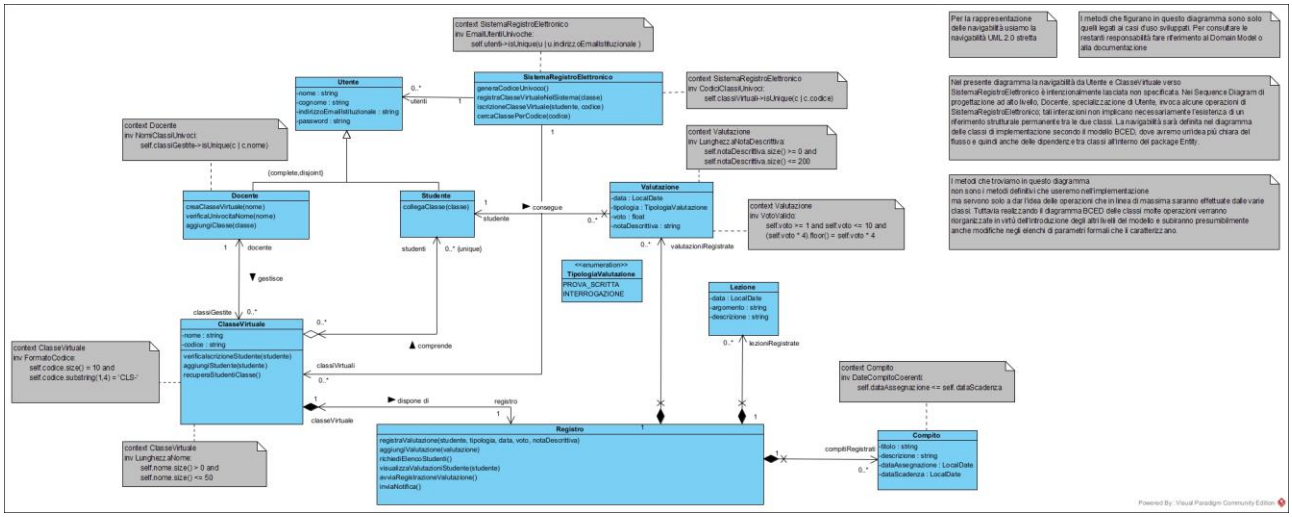
La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *"Visualizza valutazioni singolo studente*, ha fatto sorgere la necessità di definire alcuni metodi:

- Il metodo richiediElencoStudenti() della classe Registro gestisce la richiesta iniziale del docente e restituisce l'elenco degli studenti appartenenti alla classe virtuale associata al registro.
- Il metodo recuperaStudentiClasse() della classe ClasseVirtuale accede alla collezione degli studenti iscritti e la restituisce.
- Il metodo visualizzaValutazioniStudente(studente) della classe Registro coordina il recupero delle valutazioni associate allo studente indicato ricercandole nella collezione di tutte le valutazioni della classe.



2.8 Diagramma delle classi raffinato

Le aggiunte e le modifiche fatte nel corso della costruzione dei Sequence Diagrams hanno determinato lo sviluppo di un Diagramma delle Classi raffinato che riporta maggiori dettagli sugli attributi e le principali operazioni delle classi:



Innanzitutto, abbiamo aggiunto alle classi del class diagram, che sono le stesse del System Domain Model, i metodi derivanti dalla costruzione dei sequence diagram relativi solo ai casi d'uso selezionati. Per questi metodi non abbiamo specificato visibilità, tipi di ritorno né tipi dei parametri formali, perché, come detto già nei sequence, solo con la progettazione del modello BCED, introducendo i vari livelli oltre all'Entity, potremo fare scelte concrete. Per gli attributi, invece, possiamo già specificare visibilità e tipi.
Abbiamo definito il tipo enumerativo TipologiaValutazione con cui caratterizzare l'attributo "tipologia" di Valutazione.

Abbiamo inserito i seguenti vincoli in linguaggio OCL:

- Due utenti non possono avere lo stesso indirizzo email
- Il codice di una ClasseVirtuale deve avere esattamente 10 caratteri e iniziare con la sottostringa "CLS-"
- Il nome di una ClasseVirtuale deve avere lunghezza >0 e <=50 caratteri
- La nota descrittiva di una Valutazione deve avere lunghezza >=0 e <=200 caratteri (è opzionale)
- Due classi dello stesso Docente non possono avere lo stesso nome (ma due classi di docenti diversi sì)
- Due classi non possono avere lo stesso codice
- La data di assegnazione di un compito non può essere posteriore alla data di scadenza
- I voti numerici delle valutazioni devono essere compresi tra 1 e 10 con intervalli di 0.25

Inoltre, dalla realizzazione del sequence diagram del caso d'uso "Iscrizione tramite codice classe" abbiamo derivato la necessità di inserire un vincolo di univocità con "unique" alla classe Studente nell'associazione con ClasseVirtuale per indicare che uno stesso Studente non può iscriversi più volte alla stessa ClasseVirtuale.

Nella relazione di Gen/Spec tra Utente e Docente/Studente abbiamo specificato i vincoli {complete,disjoint} che indicano che ogni istanza di Utente deve essere per forza o un Docente o uno Studente: non sono ammessi utenti che non siano né uno né l'altro (complete) o tutti e due contemporaneamente (disjoint).
Sulla base non solo degli scenari dei casi d'uso selezionati, ma di tutti quelli riportati nello Use Case Diagram, abbiamo definito le navigabilità delle associazioni tra le classi.

- Docente <->ClasseVirtuale:
    - Docente->ClasseVirtuale: necessaria perché il Docente deve poter creare e gestire le proprie classi.
    - ClasseVirtuale->Docente: ogni ClasseVirtuale deve conoscere il Docente che la crea e gestisce
- ClasseVirtuale<->Studente:
    - ClasseVirtuale->Studente: ogni ClasseVirtuale deve mantenere l'elenco degli studenti iscritti
    - Studente->ClasseVirtuale: uno Studente deve conoscere le classi a cui è iscritto per poter accedere al Registro associato e consultare lezioni, valutazioni e compiti
- ClasseVirtuale<->Registro:
    - ClasseVirtuale->Registro: la ClasseVirtuale crea e possiede il proprio Registro. Questa direzione garantisce a chi accede alla classe di accedere anche al Registro per compiti, lezioni e valutazioni
    - Registro->ClasseVirtuale: questa direzione serve al registro per conoscere la classe a cui appartengono gli elementi che lo compongono con le informazioni che la caratterizzano.
- Registro->Lezione, Registro->Compito, Registro->Valutazione:
    - È il Registro che deve registrare e consultare compiti, lezioni e valutazioni. Nessuno scenario di nessun caso d'uso richiede che tali classi conoscano direttamente il Registro
- Valutazione->Studente:
    - Ogni Valutazione deve conoscere lo Studente al quale si riferisce. Ciò serve in vari scenari di casi d'uso dello UCD; ad esempio, per recuperare le valutazioni di un determinato studente per una certa ClasseVirtuale in "Visualizza valutazioni singolo studente".
      Invece Studente->Valutazione NON è necessaria perché non abbiamo bisogno per i nostri requisiti funzionali che lo Studente mantenga una collection di tutte le sue valutazioni.

La navigabilità da Utente e ClasseVirtuale verso SistemaRegistroElettronico è intenzionalmente lasciata non specificata. Nei Sequence Diagram di progettazione di alto livello, ad esempio, Docente, specializzazione di Utente, invoca alcune operazioni di SistemaRegistroElettronico; tali interazioni non implicano necessariamente l'esistenza di un riferimento strutturale permanente tra le due classi. La navigabilità sarà definita nel diagramma delle classi di implementazione secondo il modello BCED, dove avremo un'idea più chiara del flusso e quindi anche delle dipendenze tra classi all'interno del package Entity.
I metodi che troviamo in questo diagramma non sono i metodi definitivi che useremo nell'implementazione ma servono solo a dar l'idea delle operazioni che in linea di massima saranno effettuate dalle varie classi. Tuttavia, realizzando il diagramma BCED delle classi molte operazioni verranno riorganizzate in virtù dell'introduzione degli altri livelli del modello e

subiranno presumibilmente anche modifiche negli elenchi di parametri formali che li caratterizzano.
Inoltre, i metodi che figurano in questo diagramma sono solo quelli legati ai casi d'uso sviluppati. Per consultare le restanti responsabilità fare riferimento al Domain Model di partenza.
Abbiamo aggiunto i nomi alle associazioni e i ruoli alle loro estremità dove opportuno.

3. Piano di test funzionale

Si intende progettare i casi di test funzionale con la tecnica del ***Category Partition Testing***.

3.1 Crea classe virtuale

Riportiamo di nuovo una breve descrizione dello scenario del caso d'uso:
Il caso d'uso consente al Docente, dopo aver effettuato l'accesso, di creare una nuova classe virtuale specificandone il nome. Il sistema verifica che il nome sia valido, quindi che non superi i 50 caratteri e non sia già utilizzato dal docente per un'altra classe. Se tutti i controlli hanno esito positivo, genera un codice univoco, memorizza la classe e la aggiunge all'elenco delle classi del docente.
Partizione in classi di equivalenza dell'unica variabile di input sulla base della sua lunghezza.

CREA CLASSE VIRTUALE
NOME DELLA CLASSE
▪ CV1: Stringa di caratteri di lunghezza >0 e <=50 ▪ CNV1: Stringa di caratteri di lunghezza >50 **[ERROR]** ▪ CNV2: Stringa vuota **[ERROR]**

Sulla base del partizionamento della variabile di input nome classe abbiamo individuato 3 classi di equivalenza. Progettiamo un test case per CNV1 e uno per CNV2. Per CV1 abbiamo due test case, TC1 e TC2, eseguiti in due stati iniziali del sistema differenti (abbiamo infatti per essi due precondizioni diverse). Quindi complessivamente la test suite presenta 4 test case.

TEST SUITE						
Test Case ID	**Descrizione**	**Classi di equivalenza coperte**	**Pre-condizioni**	**Input**	**Output attesi**	**Post-condizioni attese**
1	Nome della classe valido	CV1: Stringa di caratteri di lunghezza >0 e <=50	Docente autenticato; **il docente non possiede una classe chiamata "3A Informatica"**	Nome = "3A Informatica"	La classe viene creata; il sistema assegna un codice univoco e la mostra nell'elenco delle classi del docente	La nuova classe è persistita e associata al docente; il codice generato non è disponibile per altre classi
2	Nome già utilizzato	CV1: Stringa di caratteri di lunghezza >0 e <=50	Docente autenticato; **il docente possiede già una classe chiamata**	Nome = "3A Informatica"	Il sistema vieta la creazione e comunica che il nome è già utilizzato	Nessuna nuova classe viene creata; lo stato delle classi del docente

			"3A Informatica"			rimane invariato
3	Nome troppo lungo	CNV1: Stringa di caratteri di lunghezza >50	Docente autenticato	Una stringa di 51 caratteri	Il sistema vieta la creazione e comunica che il nome supera la lunghezza massima consentita	Nessuna nuova classe viene creata; lo stato delle classi del docente rimane invariato
4	Nome non inserito	CNV2: Stringa vuota	Docente autenticato	Nome = ""	Il sistema vieta la creazione e comunica che è necessario inserire il nome della classe	Nessuna nuova classe viene creata; lo stato delle classi del docente rimane invariato

3.2 Iscrizione tramite codice classe

Riportiamo di nuovo una breve descrizione dello scenario del caso d'uso:
Il caso d'uso consente allo Studente, dopo aver effettuato l'accesso, di iscriversi autonomamente a una classe virtuale inserendo il relativo codice univoco. Il sistema verifica che il codice sia composto da 10 caratteri, inizi con il prefisso "CLS-", corrisponda a una classe esistente e che lo studente non sia già iscritto. Se tutte le condizioni sono soddisfatte, registra l'iscrizione e aggiunge la classe all'elenco delle classi dello studente.
Partizione in classi di equivalenza dell'unica variabile di input, codice, sulla base della sua struttura:

ISCRIZIONE TRAMITE CODICE CLASSE
CODICE
▪ CV1: Stringa di caratteri di lunghezza =10, con prefisso "CLS-" ▪ CNV1: Stringa di caratteri non vuota di lunghezza diversa da 10 **[ERROR]** ▪ CNV2; Stringa di lunghezza =10 che non inizia con "CLS-" **[ERROR]** ▪ CNV3: Stringa vuota **[ERROR]**

In virtù del partizionamento fatto per la variabile di input codice abbiamo 4 classi di equivalenza. Progettiamo un test case per CNV1, uno per CNV2 e uno per CNV3. Per CV1 invece progettiamo 3 test case, TC1, TC2 e TC3, eseguiti in 3 stati iniziali del sistema diversi (abbiamo infatti 3 precondizioni differenti). Quindi complessivamente la test suite presenta 6 test case.

TEST SUITE						
Test Case ID	**Descrizione**	**Classi di equivalenza coperte**	**Pre-condizioni**	**Input**	**Output attesi**	**Post-condizioni attese**
1	Codice valido	CV1: Stringa di caratteri di lunghezza =10, con prefisso "CLS-"	Studente autenticato ; **esiste una classe con codice "CLS-A23BC6"; lo**	Codice= "CLS-A23BC6"	Il sistema conferma l'iscrizione e mostra la classe nell'elenco delle classi	Lo studente risulta iscritto alla classe

			studente non è iscritto a quella classe		dello studente	
2	Codice inesistente	CV1: Stringa di caratteri di lunghezza =10, con prefisso “CLS-“	Studente autenticato ; **nessuna classe possiede il codice “CLS-999999”**	Codice= ”CLS-999999”	Il sistema vieta l'operazione e comunica che il codice è inesistente	Nessuna nuova iscrizione
3	Studente già iscritto	CV1: Stringa di caratteri di lunghezza =10, con prefisso “CLS-“	Studente autenticato ; **esiste la classe “CLS-123456”; lo studente è già iscritto**	Codice= “CLS-123456”	Il sistema vieta l'operazione e comunica che lo studente è già iscritto	Nessuna nuova iscrizione
4	Codice con lunghezza errata	CNV1: Stringa di caratteri non vuota di lunghezza diversa da 10	Studente autenticato	Codice= “CLS-12345”	Il sistema vieta l'operazione e comunica che il codice deve essere composto da 10 caratteri	Nessuna nuova iscrizione
5	Prefisso del codice errato	CNV2: Stringa di lunghezza =10 che non inizia con “CLS-“	Studente autenticato	Codice= “ABC-123456”	Il sistema vieta l'operazione e comunica che il codice deve iniziare con CLS-	Nessuna nuova iscrizione
6	Codice non inserito	CNV3: Stringa vuota	Studente autenticato	Codice= ""	Il sistema vieta l'operazione e comunica che è necessario inserire il codice della classe	Nessuna nuova iscrizione

3.3 Registra valutazione

Riportiamo di nuovo una breve descrizione dello scenario del caso d’uso:
Il caso d’uso consente al Docente di registrare una valutazione relativa a uno studente iscritto a una propria classe virtuale. Il docente seleziona lo studente, la tipologia di prova (scritta o interrogazione), la data, il voto e un’eventuale nota descrittiva. Il sistema verifica la validità dei dati inseriti e, in caso di esito positivo, memorizza la valutazione, conferma l’avvenuta registrazione e attiva l’invio di una notifica.

Partizione in classi di equivalenza di ogni variabile di input:

REGISTRA VALUTAZIONE				
TIPOLOGIA VALUTAZIONE	**VOTO**	**DATA**	**NOTA**	**STUDENTE**
▪ T-CV1: TIPOLOGIA=INTERROGAZIONE ▪ T-CV2: TIPOLOGIA=PROVA_SCRITTA ▪ T-CNV1: TIPOLOGIA non selezionata **[ERROR]**	*Insieme dei voti ammessi: {1, 1.25, 1.5, 1.75, 2, 2.25, 2.5, 2.75, 3, 3.25, 3.5, 3.75, 4, 4.25, 4.5, 4.75, 5, 5.25, 5.5, 5.75, 6, 6.25, 6.5, 6.75, 7, 7.25, 7.5, 7.75, 8, 8.25, 8.5, 8.75, 9, 9.25, 9.5, 9.75, 10}* ▪ V-CV1: Voto selezionato (dall'insieme dei voti ammessi scritto sopra) ▪ V-CNV1: Voto non selezionato **[ERROR]**	▪ D-CV1: Data non successiva alla data corrente e con giorno diverso da domenica ▪ D-CNV1: Data successiva alla data corrente **[ERROR]** ▪ D-CNV2: Data non successiva alla data corrente e il cui giorno è domenica **[ERROR]** ▪ D-CNV3: Data non selezionata **[ERROR]**	▪ N-CV1; Stringa di caratteri di lunghezza >=0 e <=200 ▪ N-CNV1: Stringa di caratteri di lunghezza >200 **[ERROR]**	▪ S-CV1: Studente selezionato (dallo scenario sappiamo che sarà possibile selezionare solo studenti appartenente alla classe) ▪ S-CNV1: Studente non selezionato **[ERROR]**

Abbiamo 7 classi di equivalenza non valide tra tutte le variabili di input. Considerando un test case per ogni classe non valida il numero minimo di test case che dobbiamo considerare è proprio 7. Vogliamo però considerare anche il caso di test in cui non ci sono error e tutti gli input sono validi, quindi ne aggiungiamo uno in più e consideriamo una test suite da 8 test case.

TEST SUITE						
Test Case ID	**Descrizione**	**Classi di equivalenza coperte**	**Pre-condizioni**	**Input**	**Output attesi**	**Post-condizioni attese**
1	Tutti gli input validi	T-CV1: Tipologia = INTERROGAZIONE; V-CV1: voto selezionato; D-CV1: data valida; N-CV1: nota 0–200; S-CV1: studente selezionato	Docente autenticato; classe con almeno Studente A iscritto	Studente=A; Tipologia=INTERROGAZIONE; Data=03/10/2026; Voto=7.25; Nota="Buona prova"	Il Sistema comunica l'avvenuta registrazione della valutazione	La valutazione è persistita per Studente A; viene avviato l'invio della notifica
2	Tipologia non selezionata	**T-CNV1: Tipologia non selezionata**; tutti gli altri	Docente autenticato; classe con	Studente=A; Tipologia=non selezionata; Data=03/10/2026;	Il Sistema vieta l'operazione e comunica	Nessuna valutazione viene registrata;

		input validi (V-CV1, D-CV1, N-CV1, S-CV1)	almeno Studente A	Voto=7.25; Nota="Buona prova"	che è necessario selezionare la tipologia	nessuna notifica viene inviata
3	Voto non selezionato	T-CV2: Tipologia=PROVA_SCRITTA; **V-CNV1: voto non selezionato**; tutti gli altri input validi (D-CV1, N-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=PROVA_SCRITTA; Data=03/10/2026; Voto=non selezionato; Nota="Buona prova"	Il Sistema vieta l'operazione e comunica che è necessario selezionare il voto	Nessuna valutazione viene registrata; nessuna notifica viene inviata
4	Data successiva alla data corrente	**D-CNV1: Data futura**; tutti gli altri input validi (T-CV1, V-CV1, N-CV1, S-CV1)	Docente autenticato; riferimento temporale corrente: 05/10/2026; classe con almeno Studente A	Studente=A; Tipologia=INTERROGAZIONE; Data=25/12/2027; Voto=6.5; Nota=""	Il Sistema vieta l'operazione e comunica che la data non può essere successiva a quella corrente	Nessuna valutazione viene registrata; nessuna notifica viene inviata
5	Data corrispondente a domenica	**D-CNV2: Data non futura il cui giorno è domenica**; tutti gli altri input validi (T-CV2, V-CV1, N-CV1. S-CV1)	Docente autenticato: riferimento temporale corrente: 05/10/2026; classe con almeno Studente A	Studente=A; Tipologia=PROVA_SCRITTA; Data=04/10/2026; Voto=8; Nota=""	Il Sistema vieta l'operazione e comunica che non è possibile selezionare una domenica	Nessuna valutazione viene registrata; nessuna notifica viene inviata
6	Data non selezionata	**D-CNV3: Data non selezionata**; tutti gli altri input validi (T-CV1, V-CV1, N-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=INTERROGAZIONE; Data=non selezionata; Voto=5.75; Nota="Da migliorare"	Il Sistema vieta l'operazione e comunica che è necessario selezionare una data	Nessuna valutazione viene registrata; nessuna notifica viene inviata
7	Nota troppo lunga	**N-CNV1: Nota di lunghezza >200**; tutti gli altri input validi (T-CV2, V-CV1, D-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=PROVA_SCRITTA; Data=03/10/2026; Voto=9; Nota=stringa di 201 caratteri	Il Sistema vieta l'operazione e comunica che la nota può contenere al massimo 200 caratteri	Nessuna valutazione viene registrata; nessuna notifica viene inviata
8	Studente non selezionato	**S-CNV1: Studente non selezionato**; tutti gli altri input validi (T-CV1, D-CV1, V-CV1, N-CV1)	Docente autenticato; la classe contiene almeno uno studente	Studente=non selezionato; Tipologia=INTERROGAZIONE; Data=03/10/2026; Voto=6.25; Nota=""	Il Sistema vieta l'operazione e comunica che è necessario selezionare uno studente	Nessuna valutazione viene registrata; nessuna notifica viene inviata

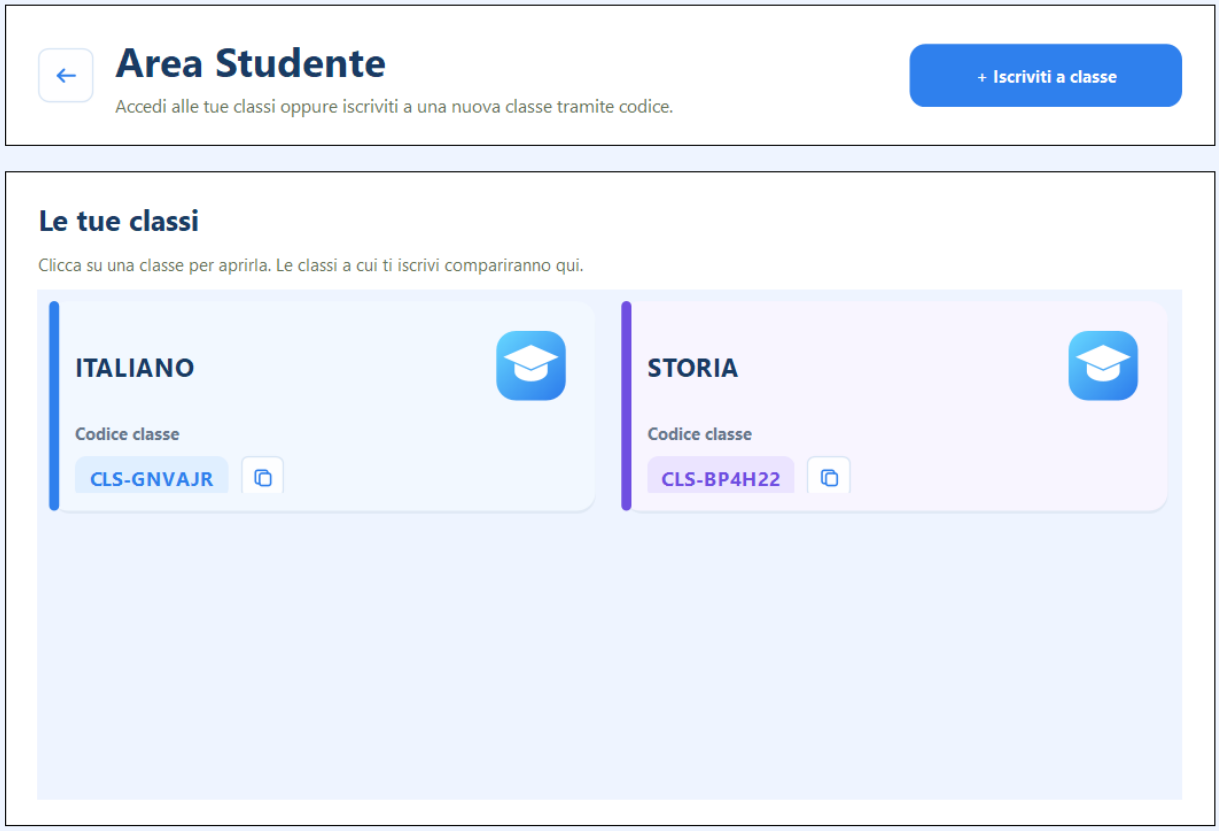
4. Progettazione

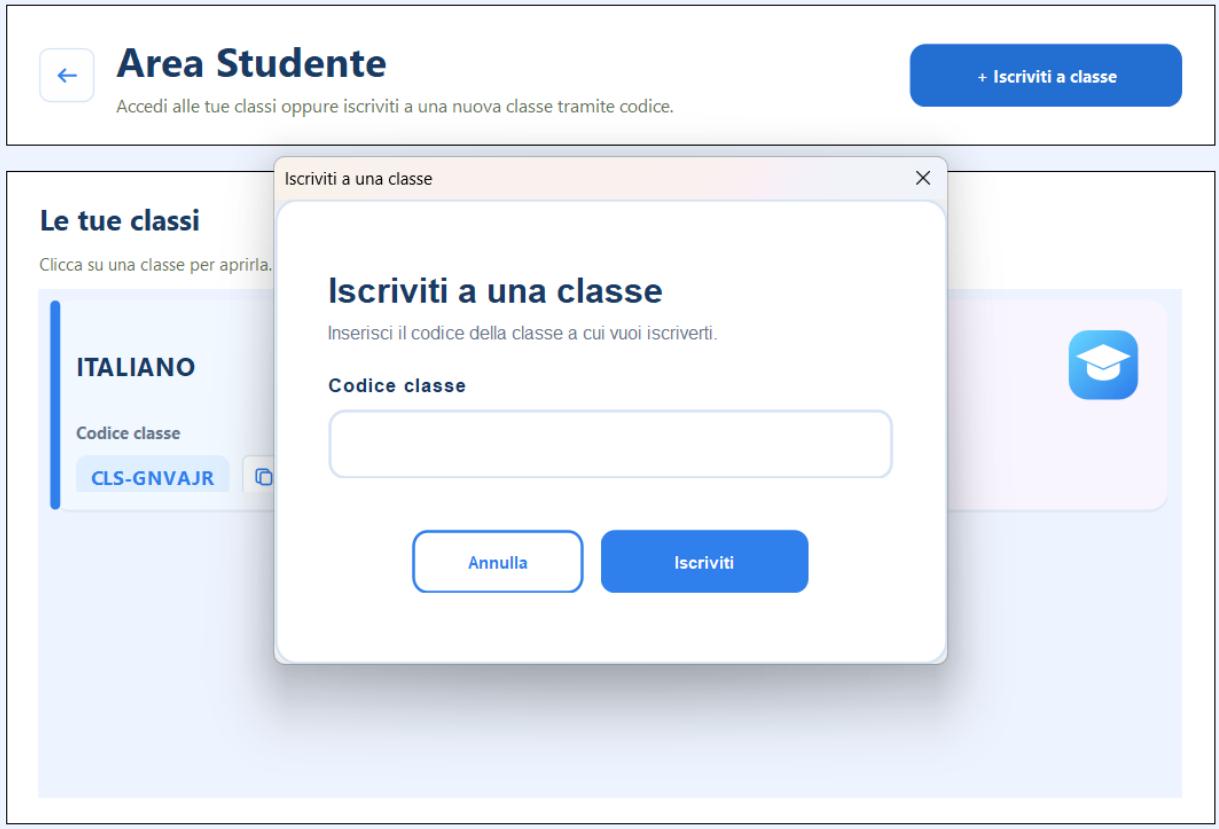
4.1 Wireframes

4.1.1 Schermata Iniziale

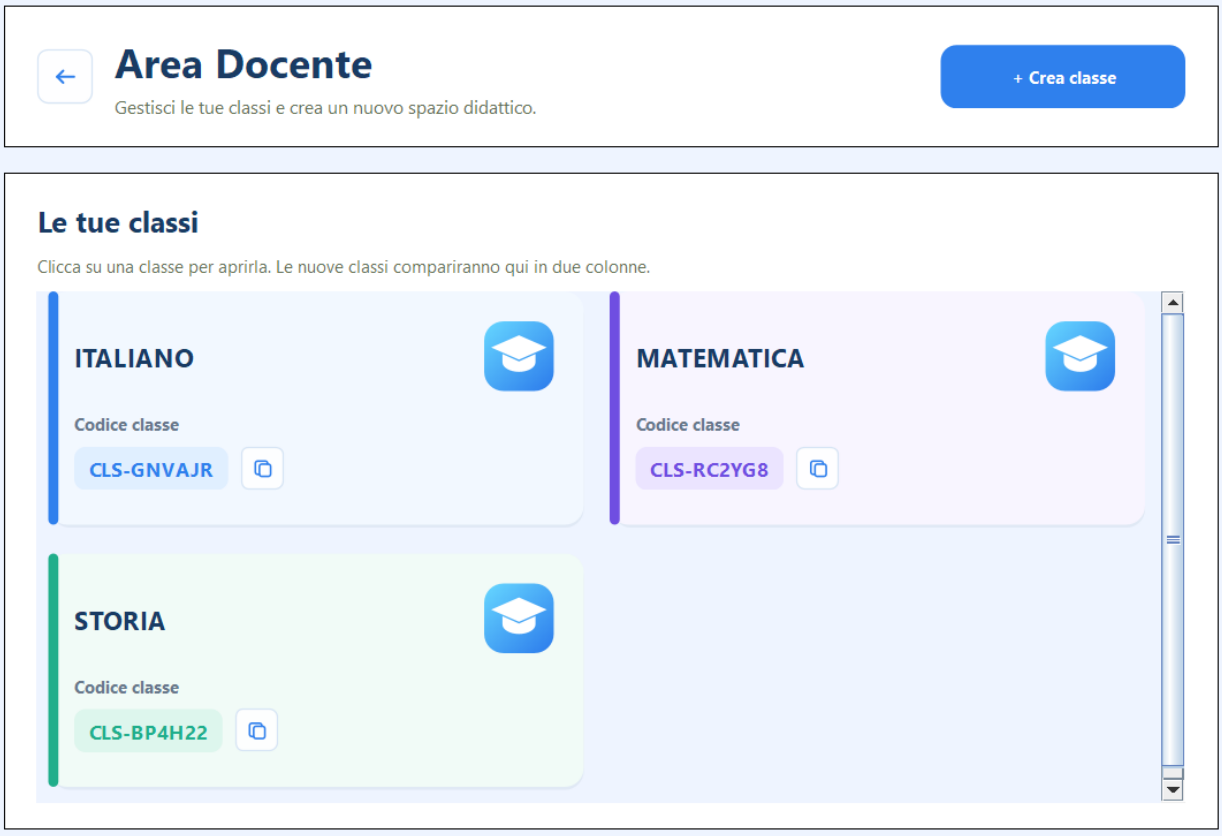


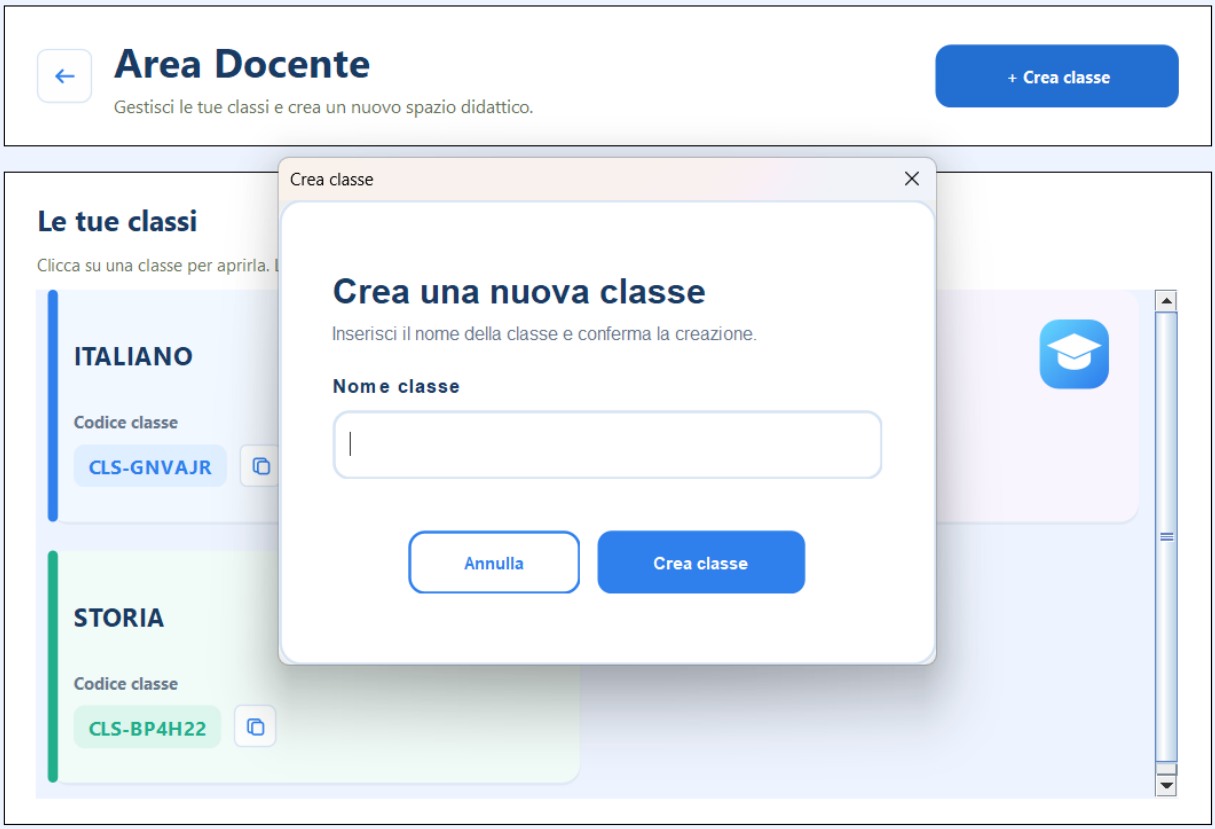
4.1.2 Schermata Studente



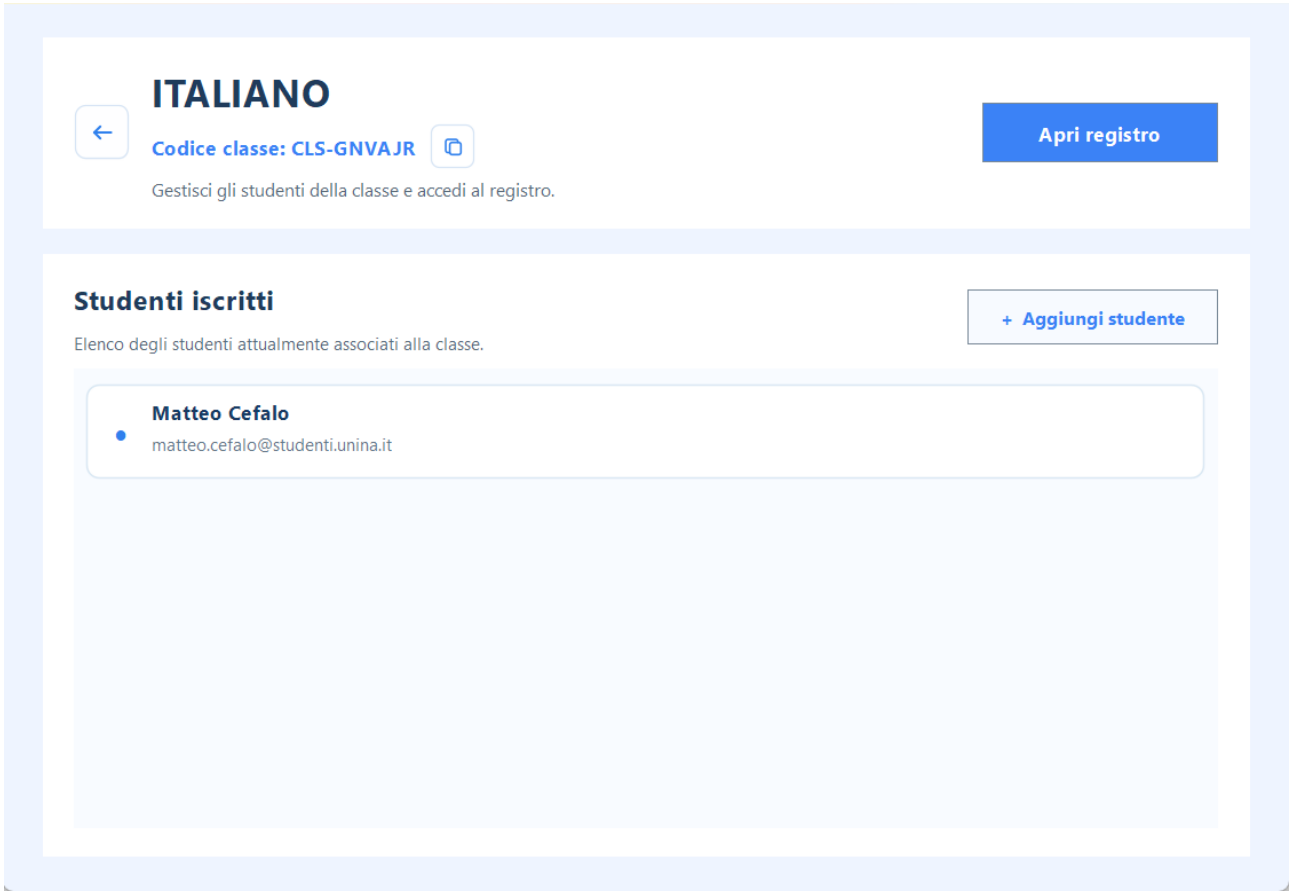


4.1.3 Schermata Docente





4.1.4 Schermata Classe



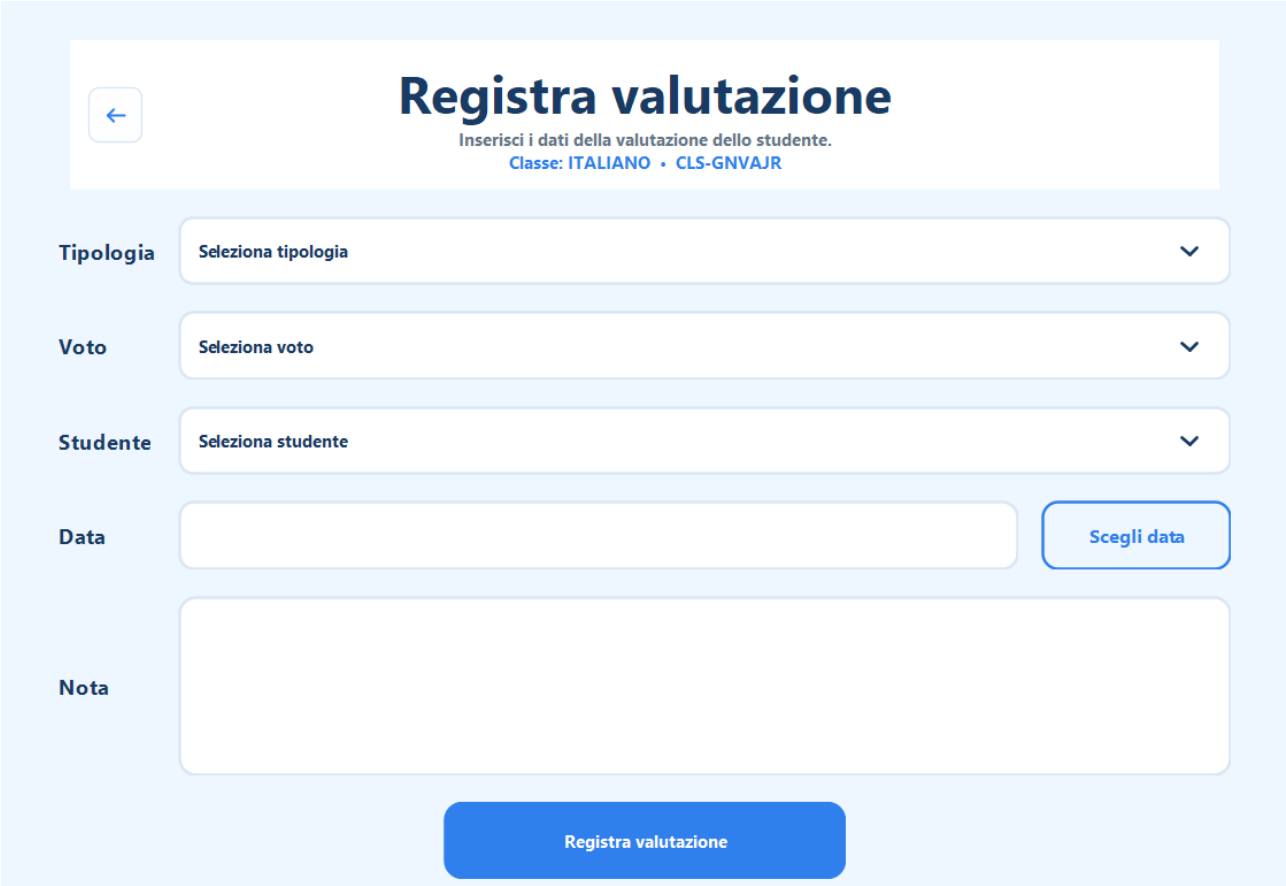
4.1.5 Schermata Registro



4.1.6 Schermata Seleziona Registra Voce



4.1.7 Schermata Registra Valutazione



4.1.8 Schermata Visualizza Info



4.1.9 Schermata Valutazioni Studenti



4.2 Diagramma delle classi

Innanzitutto, specifichiamo che noi abbiamo prodotto un'unica versione del BCED scegliendo di non partire da una bozza e poi di migliorarla utilizzando i sequence prodotti a partire da tale bozza, ma piuttosto abbiamo scelto di raffinare l'unico diagramma man mano attraverso le considerazioni emerse per mezzo dello sviluppo dei sequence dettagliati e dell'applicazione. Siamo arrivati, quindi, direttamente ad una versione definitiva del diagramma, che riportiamo e commentiamo in questa sezione.



4.2.1 Traduzione classi ed associazioni

Per realizzare il diagramma delle classi di progettazione abbiamo utilizzato il pattern architetturale BCED, con una separazione in package (Boundary, Controller, Entity, Database) delle classi sulla base delle responsabilità di ognuna. La comunicazione tra le classi, sia in termini di associazioni che di dipendenze d'uso, è stata realizzata seguendo le regole di dipendenza dei livelli del pattern BCED: *Boundary* → *Controller* → *Entity* → *Database*, in aggiunta alla classica comunicazione tra classi appartenenti allo stesso livello. In più, abbiamo

previsto anche l'inserimento del package DTO per facilitare la comunicazione tra i livelli. Tale livello comunicherà con Boundary e Controller. Abbiamo poi raggruppato le classi di eccezioni nel package Exceptions e aggiunto il package Comunicazione, contenente la classe GestoreNotifiche, per la comunicazione col sistema di notifiche esterno.
Abbiamo caratterizzato le classi del diagramma solo con metodi relativi ai casi d'uso sviluppati: "Creazione classe virtuale", "Iscrizione tramite codice classe", "Registra valutazione", "Visualizza valutazioni singolo studente". Tuttavia, la struttura generale del diagramma e le relazioni tra le classi hanno tenuto in considerazione tutti i requisiti richiesti dal committente, per cui lo sviluppo dei restanti casi d'uso dovrebbe portare semplicemente all'arricchimento delle classi già esistenti di nuovi metodi (ed eventualmente di alcuni nuovi attributi), oltre che all'aggiunta di nuove classi del Boundary rappresentanti le schermate relative alle nuove funzionalità, mantenendo invariato il resto del diagramma.
Non avendo sviluppato i casi d'uso relativi alla registrazione e all'autenticazione di docenti e studenti abbiamo previsto una soluzione temporanea per la demo dell'applicazione: abbiamo inserito una schermata iniziale in cui abbiamo sostituito i form di registrazione e autenticazione con la possibilità di selezionare il proprio ruolo: docente o studente. Quando l'utente clicca su uno dei due riquadri viene creato un utente fittizio (docente o studente a seconda della scelta) che registriamo in maniera persistente e ci permette di usare la demo dell'applicazione.

Dai testing con JUnit è emersa inoltre la necessità di trasformare la visibilità di alcuni metodi in package-private e di inserire degli ulteriori metodi, sempre con visibilità package-private, e attributi. Questa è l'unica soluzione possibile trovata per testare, senza utilizzare strumenti non visti durante il corso come la reflection o il framework mockito, i casi d'uso nel loro complesso, a partire dalle schermate del Boundary che eseguono i controlli più semplici sul formato degli input (delegando ai controller quelli più articolati), fino ad arrivare alla persistenza nel database.

Metodi la cui visibilità è passata da private a package-private ai fini del testing:

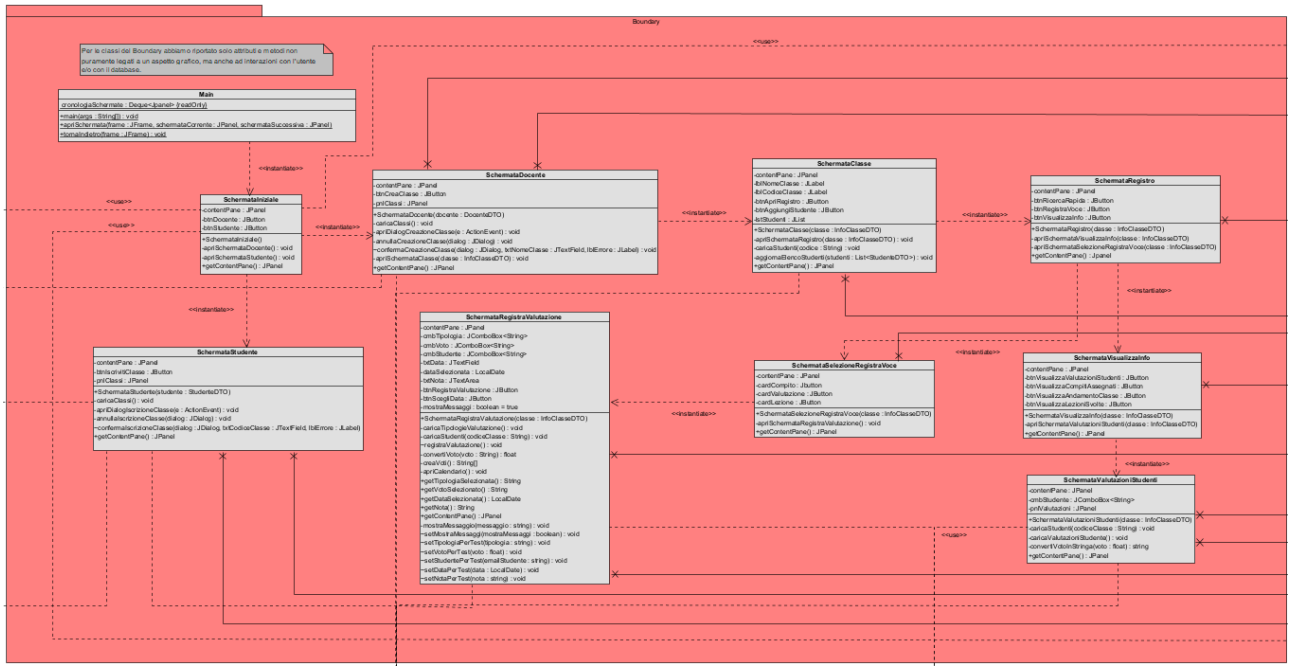
- confermaIscrizioneClasse (SchermataStudente)
- confermaCreazioneClasse (SchermataDocente)
- registraValutazione (SchermataRegistraValutazione)

Aggiunte fatte ai fini del testing:

- mostraMessaggi (attributo private di SchermataRegistraValutazione)
- mostraMessaggio (metodo private di SchermataRegistraValutazione)
- setMostraMessaggi (metodo package-private di SchermataRegistraValutazione)
- setTipologiaPerTest (metodo package-private di SchermataRegistraValutazione)
- setVotoPerTest (metodo package-private di SchermataRegistraValutazione)
- setStudentePerTest (metodo package-private di SchermataRegistraValutazione)
- setDataPerTest (metodo package-private di SchermataRegistraValutazione)
- setNotaPerTest (metodo package-private di SchermataRegistraValutazione)
- setValutazioniRegistratePerTest (metodo package-private di Registro)

4.2.2 Pattern BCED

4.2.2.1 Package Boundary



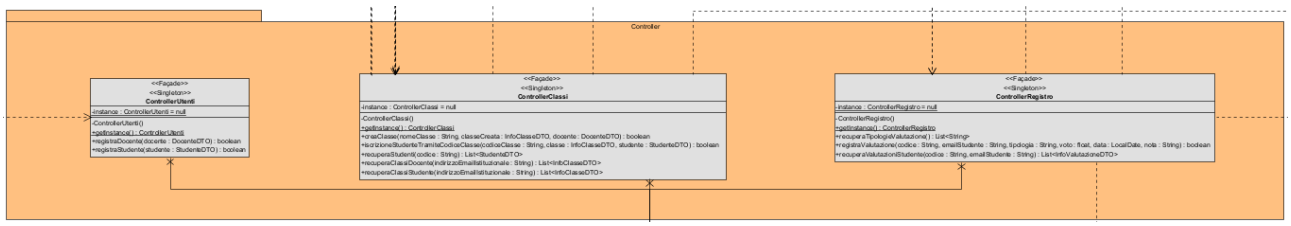
Il package Boundary contiene tutti gli oggetti responsabili della GUI e della logica di presentazione. Le interfacce sviluppate sono visibili nella sezione Wireframes della documentazione.
Per non appesantire eccessivamente il diagramma per le classi del Boundary abbiamo scelto di non riportare tutti i metodi e tutti gli attributi, evitando di raffigurare quelli puramente legati alla grafica dell'interfaccia. Quindi gli attributi e i metodi che compaiono saranno quelli che più caratterizzano l'interazione con l'utente in qualità di bottoni, handler di eventi ecc.
Le relazioni di <<istantiate>> che figurano tra una classe del Boundary e l'altra comunicano come avviene il passaggio tra le varie schermate della demo. Per esempio, nel passaggio SchermataClasse->SchermataRegistro SchermataClasse utilizza il metodo privato apriSchermataRegistro, handler dell'evento corrispondente al click del pulsante btnApriRegistro, che istanzia un oggetto SchermataRegistro e fa seguire una serie di operazioni legate al JFrame della schermata visualizzata.

Comunicazione Boundary<->DTO

In virtù della necessità di rispettare le dipendenze tra i livelli BCED, abbiamo introdotto delle classi DTO (approfondite nella sezione della documentazione relativa a tale package), che ci permettono di visualizzare nelle schermate delle interfacce le informazioni relative alle classi dell'Entity. In particolare, abbiamo previsto tra le classi del Boundary e quelle del DTO sia delle dipendenze d'uso che delle associazioni. Ad esempio, SchermataDocente recupera oggetti InfoClasseDTO relativi alle classi del docente considerato e quando apriamo la SchermataClasse in seguito al click dell'utente passiamo l'oggetto InfoClasseDTO a tale classe che lo memorizzerà in un proprio attributo. Tale oggetto sarà ora sempre passato alle schermate successive in modo

tale da mantenere sempre le informazioni relative alla classe aperta. I DTO previsti sono: DocenteDTO, InfoClasseDTO, StudenteDTO, InfoValutazioneDTO

4.2.2.2 Package Controller



Questo package contiene gli oggetti che percepiscono gli eventi generati dalle interazioni con l'interfaccia utente e ne demandano la gestione all'unico elemento del sistema software responsabile della gestione della Business Logic, il package Entity.
Abbiamo previsto 3 controller che permetteranno di gestire tutti i casi d'uso dell'applicazione e non soltanto quelli previsti dalla nostra demo: ControllerUtenti, ControllerClassi e ControllerRegistro.
Le responsabilità e i casi d'uso associati a ciascuno dei controller sono riportati nella seguente tabella. È bene precisare però che i metodi previsti dai controller non saranno solo relativi ai casi d'uso dell'UCD, ma anche ad operazioni di recupero informazioni per far sì che possano essere visualizzate nelle schermate del Boundary, come specificato anche nella tabella nella sezione "Responsabilità del controller".

Controller	**Responsabilità del controller**	**Elementi del diagramma dei casi d'uso associati al controller**
ControllerUtenti	Coordina le operazioni relative agli utenti del sistema e alle informazioni inerenti ai loro profili, in particolare la loro registrazione e autenticazione.	**Registrazione, Autenticazione** *(Non c'è un vero e proprio caso d'uso associato nel UCD, ma figura come precondizione in altri UC)*
ControllerClassi	Coordina le operazioni relative alla gestione delle classi virtuali e all'appartenenza degli utenti alle classi. Si occupa della creazione delle classi, dell'iscrizione degli studenti e del recupero delle informazioni strutturali relative alle classi.	**Creazione classe virtuale**; **Iscrizione studente ad una classe**; **Iscrizione tramite codice classe**; **Iscrizione diretta dello studente**
ControllerRegistro	Coordina tutte le operazioni relative al contenuto del registro elettronico di una classe. Gestisce la consultazione, la registrazione e la ricerca di lezioni, compiti e valutazioni, oltre al coordinamento dell'invio delle notifiche associate alla	**Visualizzazione attività classe**; **Visualizza cronologia lezioni della classe**; **Visualizza valutazioni e media per classe**; **Visualizza compiti assegnati**; **Visualizzazione registro classe**; **Visualizza valutazioni singolo studente**; **Visualizza compiti**

	registrazione di una voce di registro.	**assegnati per la classe virtuale**; **Visualizza andamento complessivo classe**; **Visualizza lezioni registrate per la classe virtuale**; **Registrazione voce di registro**; **Registra lezione svolta**; **Registra compito**; **Registra valutazione**; **Ricerca Lezioni, Compiti, Valutazioni**; **Invio notifica**

Come detto, abbiamo individuato tre sottosistemi funzionali: sottosistema relativo agli utenti, sottosistema relativo alle classi virtuali, sottosistema relativo al registro. A ciascuno di questi abbiamo associato un *Façade Controller* (GRASP Controller). La scelta di usare dei Façade Controller, che coordinano ciascuno le operazioni relative ad un intero sottosistema tra quelli menzionati, rispetto ad uno Use Case Controller per ogni caso d'uso/interazione con l'utente, si spiega molto facilmente. I casi d'uso e in generale le interazioni con l'utente sono molto numerosi e soprattutto sono facilmente raggruppabili in "gruppi funzionali". Ciò ci evita ad esempio di dover ripetere le stesse operazioni per più controller. Ad esempio, tutti gli Use Case Controller che avrebbero fatto uso del Registro avrebbero dovuto definire e implementare l'operazione recuperaRegistro() in maniera identica e lo stesso ragionamento si può fare anche con altre operazioni inerenti alle classi.

Perché usare il design pattern Façade?
Utilizzare questo design pattern ci permette di separare le classi Boundary dal dominio applicativo rappresentato dalle classi dell'Entity attraverso delle interfacce, quelle esposte dai controller, semplici e generali che indicano in maniera generica il caso d'uso/l'operazione di recupero informazioni richiesta dal Boundary. Queste interfacce nascondono la complessità interna dovuta all'insieme di tutte le operazioni che poi verranno coordinate internamente dal controller.

Perché usare il design pattern Singleton?
Abbiamo reso ControllerUtenti, ControllerClassi e ControllerRegistro dei Singleton perché ciascuno rappresenta un unico punto di coordinamento per una specifica area funzionale e non c'è utilità nel crearne più istanze contemporaneamente. Tutte le Boundary possono quindi accedere alla stessa istanza tramite il metodo getInstance() (solo la prima volta verrà effettivamente creato il controller), senza doversi preoccupare di conservare, istanziare o inizializzare ogni volta i controller.

Comunicazione Boundary<->Controller

Ogni qualvolta una classe del Boundary deve innescare un caso d'uso o un'operazione di recupero informazioni recupererà un oggetto controller, di tipo diverso a seconda della responsabilità, e chiamerà uno dei metodi esposti dall'interfaccia del controller. Con questo spieghiamo le dipendenze d'uso tra le classi del Boundary e le classi del Controller.

Comunicazione Controller<->DTO

Siccome le classi del Boundary per visualizzare le informazioni del dominio applicativo non possono ricevere oggetti dell'Entity, ma devono usare oggetti delle classi "di trasporto" del DTO,

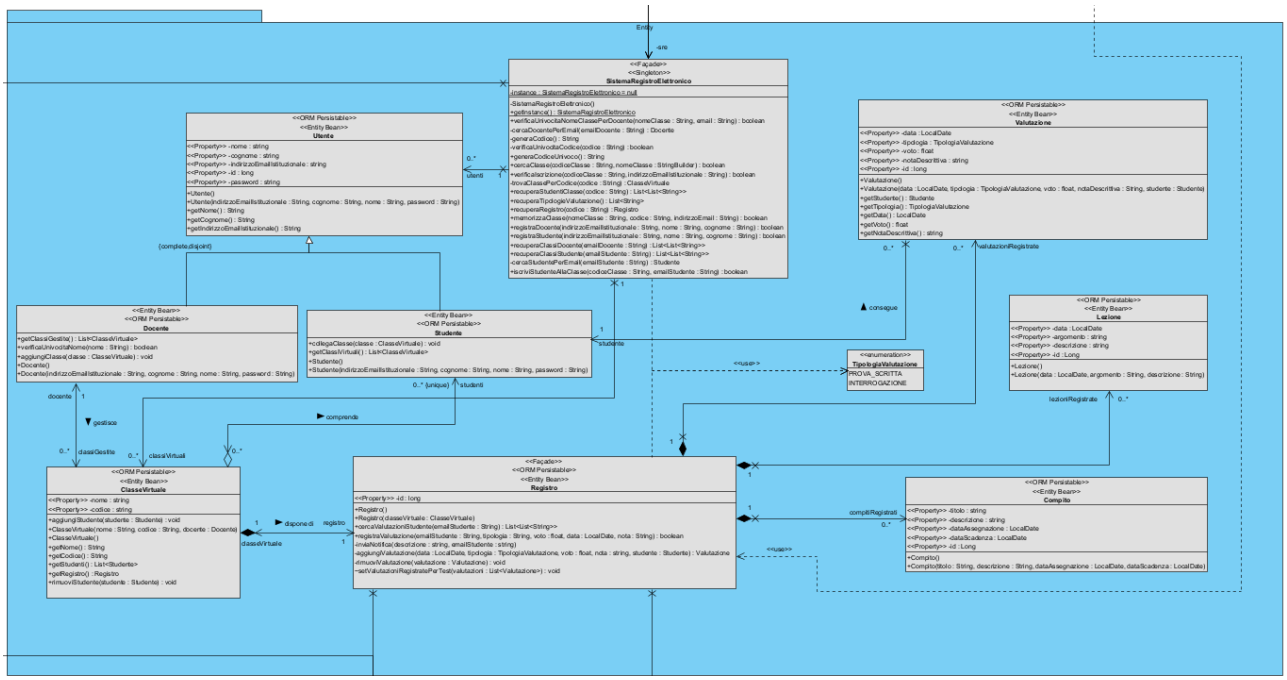


i Controller avranno delle dipendenze d'uso con le classi del package DTO perché spesso avranno il compito di istanziarle e restituirle alle classi del Boundary, oppure avranno il compito di ricevere un oggetto DTO dal Boundary e da esso estrapolare le informazioni che gli servono per interfacciarsi con opportuni elementi del dominio applicativo.

Precisazione su ControllerUtenti

Per quanto riguarda ControllerUtenti sappiamo che è responsabile delle operazioni relative agli utenti, quali registrazione e autenticazione. Nella nostra applicazione demo però abbiamo detto che questi casi d'uso non sono sviluppati e sono sostituiti con la registrazione di un docente o utente fittizio a seconda della schermata selezionata. Quindi questo Controller si occuperà proprio di coordinare queste chiamate del Boundary della nostra demo, tramite i metodi registraStudente() e registraDocente(). In ogni caso i suoi legami con le altre classi e la sua struttura qualora si scegliesse di sviluppare propriamente quei due casi d'uso dovrebbero subire al più pochi cambiamenti.

4.2.2.3 Package Entity



Il Package Entity contiene tutte le classi che rappresentano le entità del dominio applicativo e corrispondono alle strutture dati presenti all'interno del database di persistenza.

Innanzitutto, notiamo che la base di partenza del package Entity del nostro diagramma BCED l'abbiamo derivata dal precedente diagramma delle classi. In tale diagramma, infatti, avevamo definito già le relazioni tra le classi del dominio applicativo che sono rimaste immutate, eccetto per le associazioni: SistemaRegistroElettronico<->Utente, SistemaRegistroElettronico<->ClasseVirtuale. Infatti, nel precedente diagramma delle classi avevamo lasciato indefinita la navigabilità dal lato di SistemaRegistroElettronico, non potendo ancora scegliere se ClasseVirtuale e Utente avrebbero dovuto conservare un attributo di tale classe o meno. Ora, invece, avendo chiarito meglio il flusso delle chiamate rispetto alle precedenti valutazioni in fase di analisi possiamo scegliere di avere unidirezionali tali associazioni; infatti, come ora vedremo SistemaRegistroElettronico sarà un singleton che si comporterà da Façade del dominio applicativo comunicando direttamente col livello Controller. Saranno quindi le classi

del livello controller a chiamare i metodi della sua interfaccia, riassuntivi delle operazioni da invocare sulle classi dell'Entity, e quindi non c'è bisogno che altre classi dell'Entity ne memorizzino un riferimento.

Ovviamente rispetto al precedente diagramma possiamo ora stabilire con certezza attributi delle classi con visibilità e tipo, metodi con visibilità, parametri formali(ciascuno col tipo opportuno) e tipo di ritorno.

Comunicazione Controller<->Entity e Pattern Façade

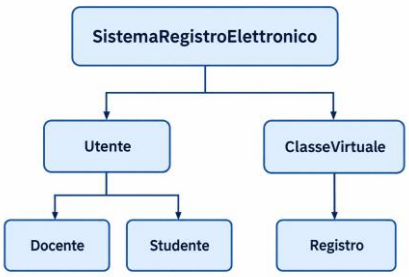
La comunicazione tra il livello Controller e il livello Entity è veicolata attraverso due punti di accesso del dominio applicativo per i controller: SistemaRegistroElettronico e Registro. I Controller hanno tutti un attributo SistemaRegistroElettronico (che, come vedremo a breve, è un singleton) e ControllerRegistro ha in più una dipendenza d'uso con Registro.

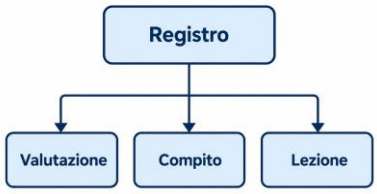
Per entrambe le classi usiamo il design pattern Façade perché permettono con i metodi che espongono di semplificare le chiamate dei controller. In questo modo riassumiamo le diverse operazioni che andranno fatte sulle classi dell'Entity per completare un caso d'uso in poche operazioni chiamate su questi due Façade che poi nella loro logica interna si occuperanno di interfacciarsi con le altre classi del package Entity. SRE e Registro, infatti, grazie alle relazioni che le collegano alle altre classi dell'Entity possono svolgere questo ruolo con estrema facilità. Vediamo meglio perché.

SistemaRegistroElettronico è la Facade del dominio complessivo: conosce gli utenti, le classi virtuali e permette ai controller di accedere agli oggetti corretti senza dover navigare direttamente tutte le Entity. Per esempio, invece di far cercare a ControllerClassi un Docente nella lista degli utenti, o a ControllerRegistro una classe e poi il suo registro, deleghiamo queste operazioni a SRE.

Registro, invece, rappresenta il punto di accesso al sottodominio delle attività didattiche di una singola classe. Per esempio, ControllerRegistro non dovrebbe cercare manualmente una valutazione dentro una lista, costruire direttamente tutte le relazioni o conoscere come sono organizzate lezioni e compiti. Recupera il Registro corretto e poi gli delega queste operazioni.

Dietro SistemaRegistroElettronico e Registro si "nascondono":





Pattern Singleton

Oltre al pattern Façade, per SistemaRegistroElettronico adottiamo anche il pattern singleton. Il motivo è che SistemaRegistroElettronico rappresenta l'intero sistema applicativo e perciò dal punto di vista logico, durante l'esecuzione dell'applicazione vogliamo avere un solo sistema centrale, che mantenga il riferimento allo stato generale. Infatti, proprio alla luce di ciò, la prima volta che viene costruito SistemaRegistroElettronico viene caricato direttamente con i dati memorizzati in maniera persistente perché deve da subito rappresentare lo stato corrente dell'intero sistema applicativo.

Tra l'altro questa scelta si abbina perfettamente alla scelta di fare Singleton i Controller. Ciascuno dei tre controller, infatti, è un singleton che mantiene il singleton del SistemaRegistroElettronico come attributo.

Registro, invece, rappresenta il registro di una singola ClasseVirtuale, pertanto devono esistere più istanze di Registro, una per ogni ClasseVirtuale, che dovranno contenere informazioni diverse. Rendere Registro singleton e prevedere quindi una sola istanza di tale classe per tutto il sistema significherebbe che lezioni, compiti e valutazioni di classi diverse confluirebbero nello stesso oggetto Registro e questo non va bene. Infatti, abbiamo preferito che ControllerRegistro recuperi attraverso SistemaRegistroElettronico il Registro associato alla classe di interesse e su di esso chiami i metodi. Non avrebbe senso memorizzarlo come attributo del controller, che è un singleton, visto che il registro varia a seconda della classe selezionata.

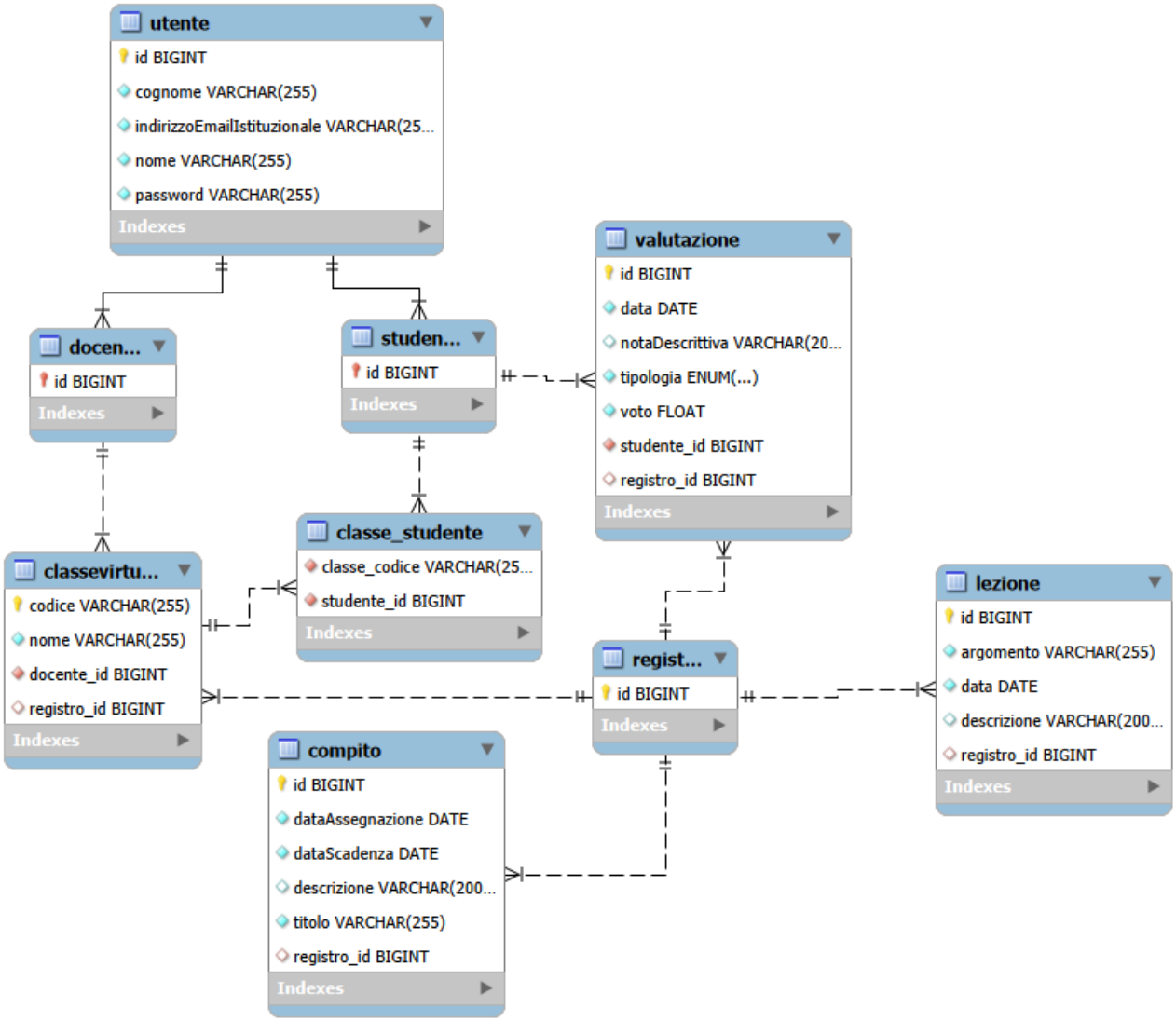
Stereotipi aggiunti alle classi dell'Entity

Alcune classi di dominio individuate nel precedente diagramma delle classi sono confluite nel package Entity come Entity Bean marcate <<ORM Persistable>>: i loro attributi sono stati marcati con lo stereotipo <<Property>> ed è stato aggiunto a ciascuna (ad eccezione di ClasseVirtuale per la quale abbiamo usato come chiave primaria il codice univoco già in suo possesso) un identificatore (id : Long) che fungerà da chiave primaria della tabella associata nel database relazionale (come è possibile vedere dal modello E-R riportato nella prossima sezione sul package Database).

In particolare, le Entity che abbiamo reso persistenti sono: Utente, Docente, Studente, ClasseVirtuale, Compito, Lezione, Valutazione, Registro.

Per vedere le annotazioni di jakarta persistence inserite nel codice java per il funzionamento di Hibernate si rimanda alla sezione di Implementazione.

4.2.2.4 Package Database



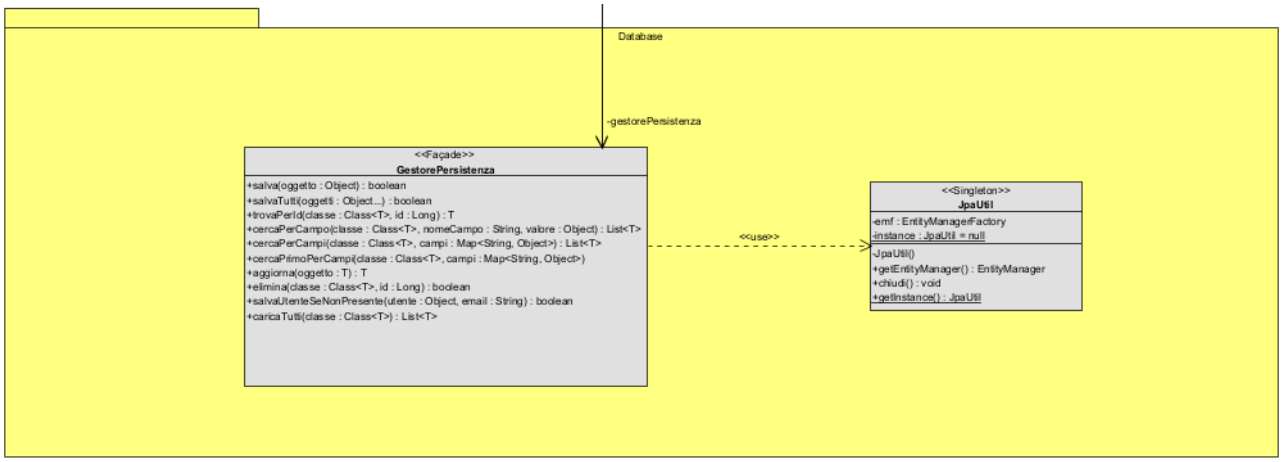
Il package database contiene le classi responsabili dell'accesso ai dati persistenti, rendendo indipendente le classi della Business Logic (Entity) dalla tecnologia di persistenza utilizzata.

Abbiamo scelto di non adottare un DAO specifico per ogni classe dell'Entity per evitare duplicazione di codice e dipendenze troppo forti tra il livello Entity e il livello Database.
Infatti, se usiamo DAO specifici per ogni Entity oltre a ripetere lo stesso codice in ognuno avremmo che il package Database conoscerebbe le classi del package Entity e violeremmo le regole di dipendenza tra i livelli del BCED. Preferiamo allora introdurre un'unica classe GestorePersistenza che esponga metodi generici che facciano uso del polimorfismo con Object e dei tipi generici. In questo modo evitiamo la ripetizione dello stesso codice in più classi ed evitiamo che classi del package Database conoscano quelle del package Entity, cioè evitiamo così la creazione di dipendenze cicliche. Inoltre, con questa soluzione rendiamo più semplice apportare modifiche al package Database perché va modificata una sola classe, GestorePersistenza, anziché le varie StudenteDAO, DocenteDAO, ClasseVirtualeDAO ecc.

Dunque, GestorePersistenza applica il pattern Facade esponendo un'interfaccia CRUD generica e indipendente dal dominio (salva, trovaPerId, cercaPerCampo, ecc). JpaUtil ha la responsabilità

di centralizzare la gestione della EntityManagerFactory, mentre GestorePersistenza utilizza l'EntityManager restituito dall'apposito metodo di JpaUtil.
JpaUtil è Singleton proprio perché incapsula la EntityManagerFactory, risorsa costosa che deve essere inizializzata una sola volta e condivisa da tutte le operazioni di persistenza.

Precisazione legata alla nostra demo (metodo salvaUtenteSeNonPresente() non figurerà nell'applicazione completata)
Abbiamo inserito in GestorePersistenza anche un metodo, salvaUtenteSeNonPresente(), che in realtà non figurerebbe nella versione completa dell'applicazione con lo sviluppo di tutti i casi d'uso. Questo metodo è legato alla registrazione fittizia di un docente e uno studente per utilizzare la demo. In particolare, se avessimo sviluppato i casi d'uso Registrazione e Autenticazione, non avremmo avuto bisogno di questo metodo perché ad un utente già registrato sarebbe stato impedita una seconda registrazione e permessa invece l'autenticazione. Nel nostro caso, invece, registriamo ogni volta all'avvio dell'applicazione lo stesso utente fittizio. Questo comporta che se usassimo il semplice metodo salva() di GestorePersistenza riceveremmo un errore dal database per ogni utilizzo successivo della demo perché l'utente appare già registrato. Per risolvere ciò bisognerebbe o cancellare ogni volta il database, ma questo non sarebbe fruttuoso per l'utilizzo della demo, oppure, ed è la scelta che abbiamo preso, bisognerebbe usare un metodo come salvaUtenteSeNonPresente(), che simula comunque una registrazione e fa in modo di non dare errore se l'utente fittizio risulta già registrato.



4.2.2.5 Package Exceptions

Nel diagramma BCED figura anche il package Exceptions, che creiamo per ospitare le 3 classi di eccezioni che sono sollevate nei casi d'uso che abbiamo sviluppato. Si rimanda alla sezione di implementazione per approfondimenti sul significato di queste eccezioni. In particolare, a comunicare con questo package sono il package Controller e il package Boundary. Le classi del Controller che lanciano queste eccezioni (per i casi d'uso sviluppati è ControllerClassi) avranno una dipendenza d'uso con stereotipo <<istantiate>> con le classi rappresentative delle eccezioni del package Exceptions. Le classi del Boundary, invece, che fanno il catch di tali eccezioni avranno con tali classi una dipendenza d'uso con stereotipo classico di <<use>>.

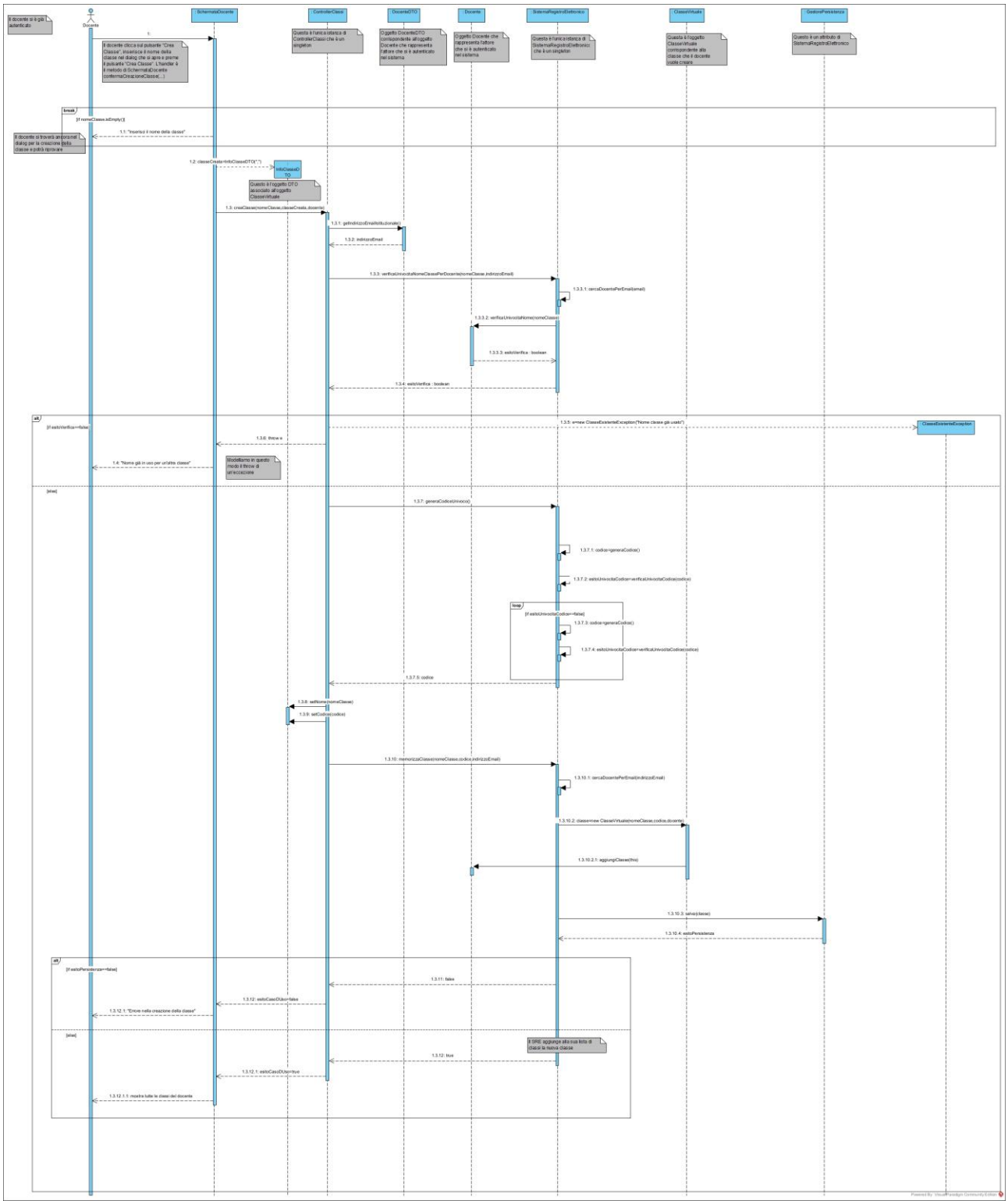
4.2.2.6 Package Comunicazione

Aggiungiamo il package Comunicazione per gestire la comunicazione col sistema di notifiche esterno, con un ruolo che è essenzialmente equivalente a quello del package Database per la comunicazione col database per la persistenza dei dati. Questo package permette di rispettare il principio di separazione delle responsabilità e in particolare permette di isolare la responsabilità legata all'invio delle notifiche dalla logica applicativa del sistema. Al suo interno abbiamo introdotto la classe GestoreNotifiche dedicata a tale funzionalità.

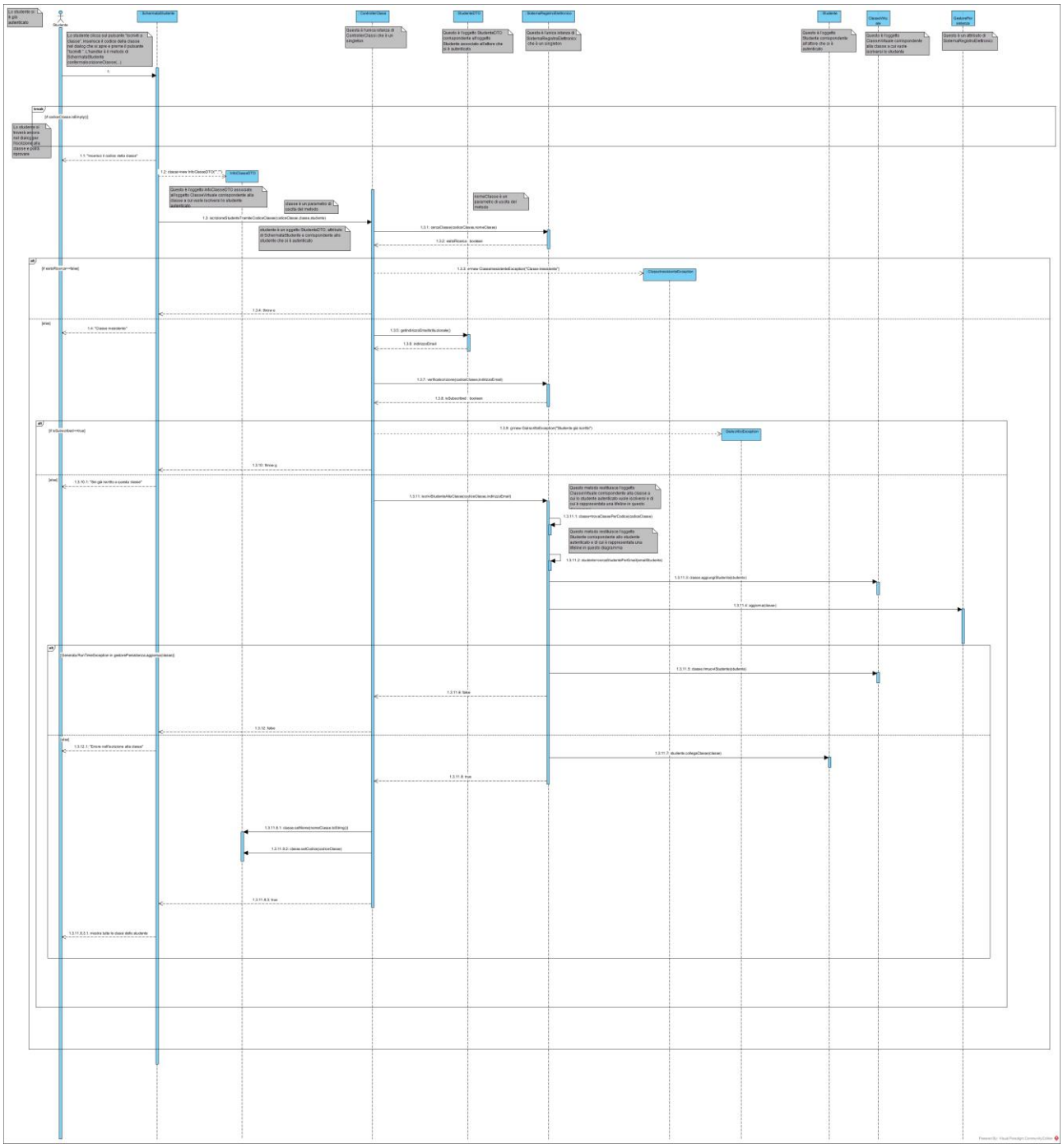
4.3 Diagrammi di sequenza

Si riportino di seguito i diagrammi di sequenza di progetto per il/i casi d'uso sviluppati fino alla codifica in Java.

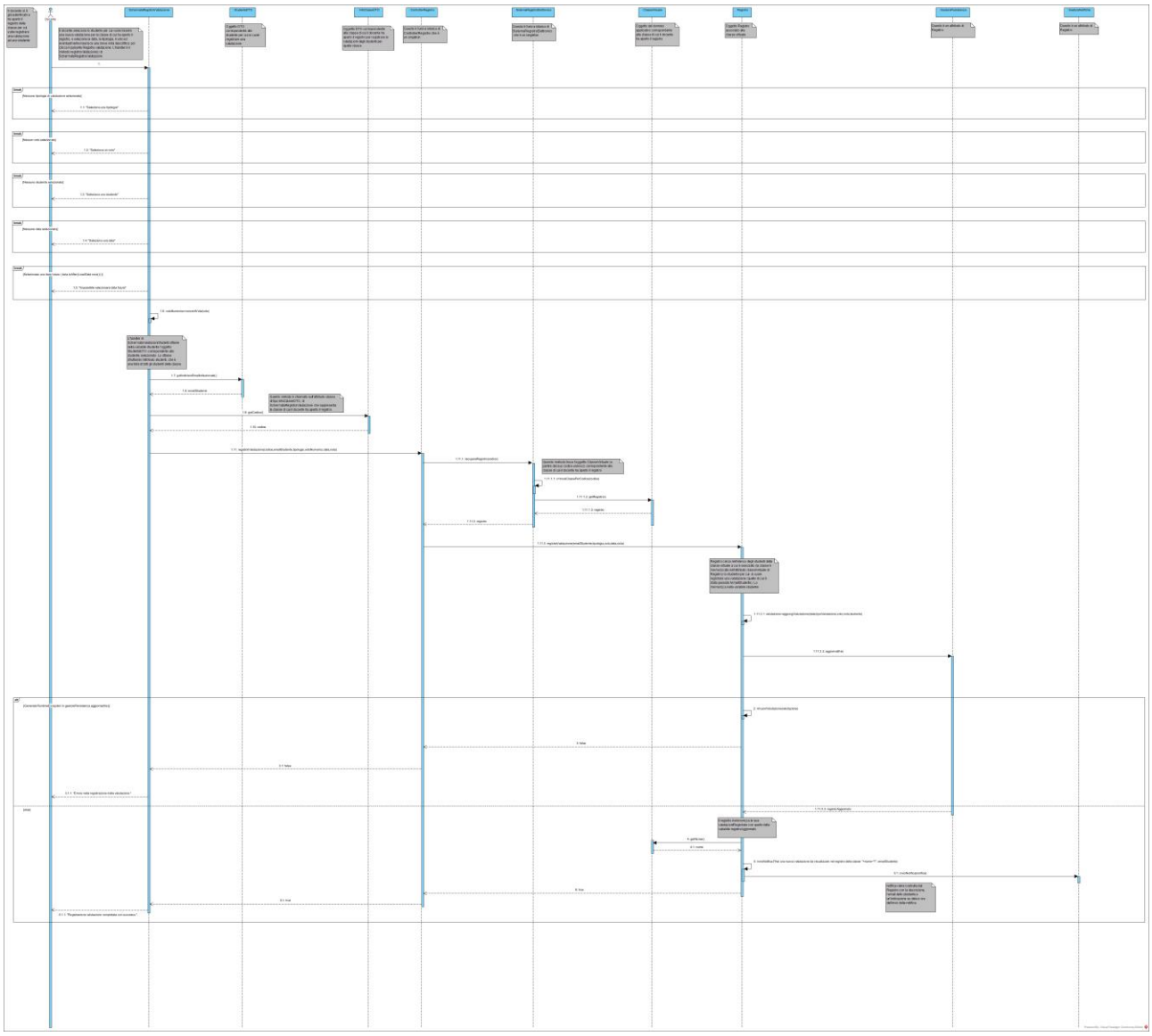
4.3.1 Creazione classe virtuale



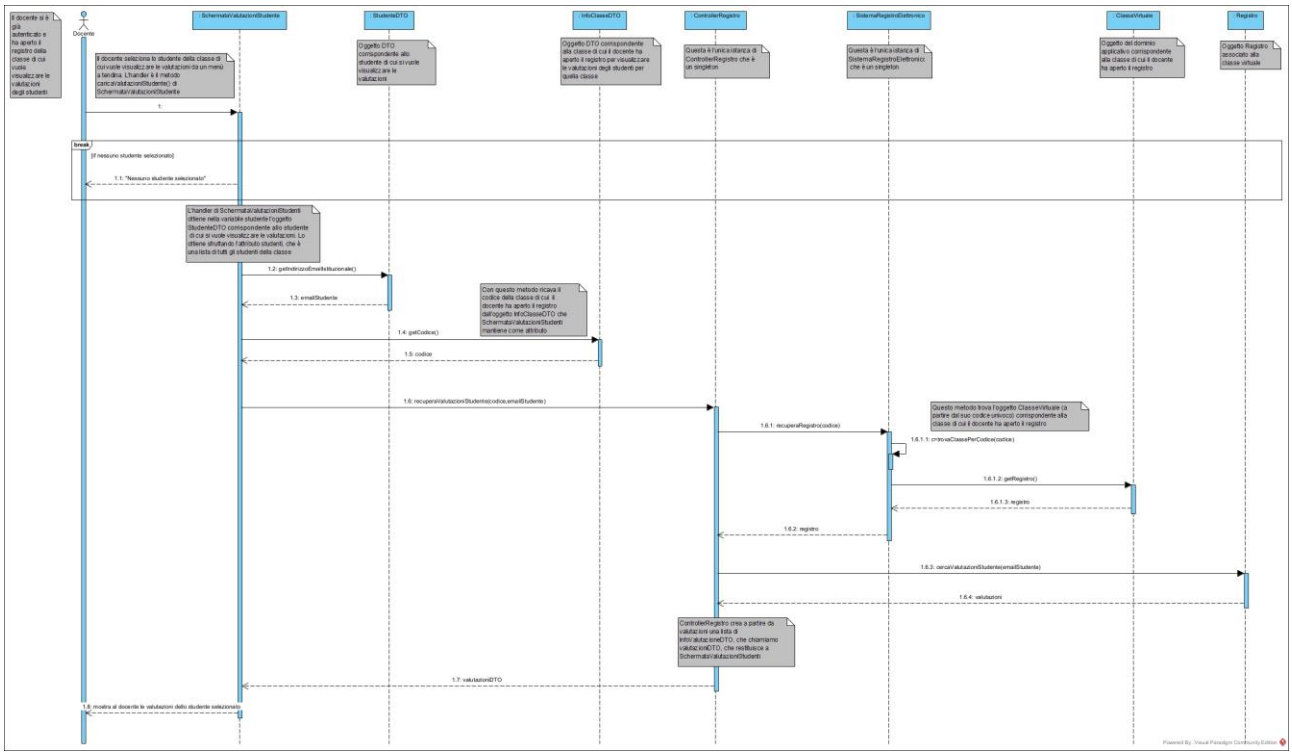
4.3.2 Iscrizione tramite codice classe



4.3.3 Registra Valutazione



4.3.4 Visualizza valutazioni singolo studente



5. Implementazione

Il sistema è stato realizzato utilizzando il linguaggio di programmazione Java (23). Per la realizzazione delle interfacce grafiche del Boundary abbiamo utilizzato il plugin Swing UI Designer di IntelliJIDEA, basato sul framework Swing. Per gestire i dati in maniera persistente abbiamo utilizzato MySQL come DBMS relazionale di riferimento e il framework ORM Hibernate che sfrutta il driver JDBC di MySQL. L'architettura software è stata realizzata seguendo il pattern BCED con la suddivisione delle classi in package: Boundary, Controller, Entity, Database (con l'aggiunta dei package DTO, Exceptions e Comunicazione).

5.1 Package Database

Classi:

- GestorePersistenza
- JpaUtil

Librerie importate:

- jakarta.persistence
- java.util

5.2 Package Entity

Classi:

- ClasseVirtuale
- Compito
- Lezione
- Valutazione
- Registro
- Studente
- Docente
- Utente (generalizzazione di Docente e Studente)
- SistemaRegistroElettronico

Enumerativi:

- TipologiaValutazione

Librerie:

- jakarta.persistence
- java.util
- java.time

Spiegazione delle annotazioni jakarta utilizzate per le classi dell'Entity.

Innanzitutto, ricordiamo le classi dell'Entity che abbiamo scelto di rendere persistenti: ClasseVirtuale, Compito, Docente, Lezione, Registro, Studente, Utente, Valutazione.

Annotazioni sulle classi

L'annotazione @Entity dice a Hibernate che la classe con tale annotazione è una classe persistente e quindi va memorizzata nel database.

Vediamo invece come abbiamo scelto di gestire la gerarchia Utente, Docente, Studente. La scelta che abbiamo preso è innanzitutto di creare 3 tabelle separate. Rappresenteremo un Docente o uno Studente con una riga nella tabella Utente e una riga nella corrispondente tabella della sottoclasse usando lo stesso valore per la chiave primaria id. Infatti, in Docente e in Studente l'attributo id non è solo chiave primaria, ma anche chiave esterna verso la tabella della superclasse. Per realizzare ciò abbiamo annotato la classe utente sia con @Entity che con @Inheritance(strategy = InheritanceType.JOINED). Abbiamo annotato poi Docente e Studente oltre che con @Entity anche con @PrimaryKeyJoinColumn(name = "id"). "JOINED" dice ad Hibernate di creare 3 tabelle separate collegate tra loro, mentre l'annotazione che usiamo per Docente e Studente dice "nella tabella di questa sottoclasse, usa la colonna id come chiave primaria e come colonna di collegamento alla PK della superclasse".

In questo modo realizziamo correttamente tra l'altro una relazione di gen/spec complete e disjoint come indicato nel diagramma delle classi tra Docente, Studente e Utente.

Annotazioni sugli attributi che non sono legati ad associazioni con altre classi

Ricordiamo che l'annotazione @Id stabilisce che l'attributo con tale annotazione sarà la chiave primaria della tabella corrispondente nel database. Ad eccezione di ClasseVirtuale per cui abbiamo annotato con @Id l'attributo codice perché può svolgere perfettamente il ruolo di chiave primaria, per tutte le altre classi abbiamo aggiunto un attributo id di tipo Long, annotato con @Id per far presente che sarà chiave primaria della tabella e con @GeneratedValue(strategy = GenerationType.IDENTITY) che dice che il valore sarà generato dal DBMS automaticamente al momento dell'INSERT nel database, con una colonna AUTO_INCREMENT.

Ricordiamo che l'annotazione @Column significa che quell'attributo dell'entity viene mappato su una colonna della tabella corrispondente alla classe nel database. Per sostanzialmente tutti gli attributi delle classi che non siano derivanti da associazioni con altre classi abbiamo scritto quindi @Column(nullable=false) per indicare che quell'attributo è mappato su una colonna del database che non può contenere valori NULL, tranne notaDescrittiva di Valutazione, descrizione di Compito e descrizione di Lezione che invece sono parametri opzionali e per cui quindi non mettiamo il nullable. In alcuni casi abbiamo impostato anche la lunghezza massima dei valori che possono essere contenuti con @Column(length=_valore_), per esempio con notaDescrittiva di Valutazione, descrizione di Compito e descrizione di Lezione. Per l'attributo indirizzoEmailIstituzionale di Utente abbiamo l'annotazione @Column(nullable=false, unique=true). "unique=true" serve per dire che i valori di quella colonna della tabella devono essere unici (questo è sottinteso per le colonne corrispondenti alle chiavi primarie, cioè quelle corrispondenti agli attributi annotati con @Id). L'attributo tipologia di Valutazione ha tipo TipologiaValutazione che è un enumerativo. È etichettato dunque non solo con la classica annotazione @Column(nullable=false), ma anche con @Enumerated(EnumType.STRING) che serve per gestire il tipo enumerativo. In particolare, dice a Hibernate di salvare nel DB "PROVA_SCRITTA" e "INTERROGAZIONE" come stringhe.

In Registro gli attributi gestorePersistenza e gestoreNotifiche sono annotati con @Transient. Questa annotazione indica che Hibernate deve ignorare completamente questi attributi durante la persistenza. Infatti, noi non vogliamo rendere gestorePersistenza e gestoreNotifiche persistenti in memoria, non vogliamo cioè che siano delle colonne della tabella del database corrispondente al Registro, li manteniamo come attributi in Registro solo perché questo rappresenta il collegamento tra il package Entity e il package Database e tra il package Entity e il package Comunicazione.



Annotazioni sugli attributi che sono legati ad associazioni con altre classi

Imposteremo sempre esplicitamente fetch=FetchType.EAGER (anche se in alcuni casi sarebbe sottointeso automaticamente) per dire ad Hibernate che quando carica un oggetto di una delle due classi coinvolte nell'associazione deve immediatamente caricare anche gli oggetti dell'altra classe dell'associazione ad esso correlati.

Per esempio, consideriamo la relazione: Docente 1 -------- 0…* ClasseVirtuale che è un esempio di come gestiremo le relazioni 1 a molti. Un docente può gestire più classi, mentre ogni classe ha un solo docente.

```
@OneToMany(
   mappedBy = "docente",
   fetch = FetchType.EAGER
)
private List<ClasseVirtuale> classiGestite;
```

mappedBy="docente" è molto importante perché serve a dire che Docente non è il lato proprietario della relazione JPA, ma lo è ClasseVirtuale con l'attributo docente perché lì abbiamo la @JoinColumn.

Infatti, se guardiamo ClasseVirtuale abbiamo:

```
@ManyToOne(
   optional = false,
   fetch = FetchType.EAGER
)
@JoinColumn(
   name = "docente_id",
   nullable = false
)
private Docente docente;
```

@ManyToOne perché molte classi possono appartenere allo stesso docente.

optional=false dice a JPA che non può esistere una ClasseVirtuale senza docente

@JoinColumn(name="docente_id") dice che nella tabella ClasseVirtuale deve esserci una foreing key, docente_id, che punta a Docente.

nullable=false rafforza a livello database il concetto di optional=false

Con optional=false operiamo a livello di relazione tra oggetti nel modello, cioè imponiamo che una ClasseVirtuale debba avere necessariamente un Docente. È un vincolo sul modello a oggetti. Con nullable=false, invece, operiamo sulla colonna del database e cioè diciamo che la colonna della foreign key non può contenere valori null.

Vediamo ora come abbiamo gestito Studente 0…* -------- 0...* ClasseVirtuale per capire come abbiamo gestito una molti a molti. Uno studente può iscriversi a più classi e una classe può contenere più studenti.

Lato studente abbiamo:

```
@ManyToMany(
   mappedBy = "studenti",
   fetch = FetchType.EAGER
)
private List<ClasseVirtuale> classiVirtuali;
```

mappedBy = "studenti" dice che Studente è il lato inverso perché è mappato da studenti nell'altro lato della relazione. Il lato proprietario, infatti, sarà ClasseVirtuale con l'attributo studenti, dove infatti troviamo @JoinTable().

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
   name = "classe_studente",
   joinColumns =
      @JoinColumn(name = "classe_codice"),
   inverseJoinColumns =
      @JoinColumn(name = "studente_id")
)
private List<Studente> studenti;
```

Una relazione molti a molti come questa richiede una tabella intermedia come ben sappiamo.

Con @JoinTable(name = "classe_studente") stabiliamo il nome di questa tabella.

joinColumns (name = "classe_codice") rappresenta e dà il nome alla foreign key della tabella intermedia verso l'Entity proprietaria, cioè ClasseVirtuale; mentre inverseJoinColumns(name= "studente_id") rappresenta e dà il nome alla foreign key verso l'altra Entity, cioè Studente. Le due foreign key formano insieme la primary key della tabella "classe_studente".

Analizziamo la relazione tra ClasseVirtuale e Registro. È una composizione uno a uno.

Lato ClasseVirtuale abbiamo:

```
@OneToOne(
```




```
    cascade = CascadeType.ALL,
    orphanRemoval = true,
    fetch = FetchType.EAGER
)
@JoinColumn(
    name = "registro_id",
    unique = true
)
private Registro registro;
```

@OnetoOne rappresenta la cardinalità uno a uno.

Cascade=CascadeType.ALL è necessario per tradurre la semantica di composizione. Infatti CascadeType.ALL comprende operazioni quali PERSIST, MERGE, REMOVE e prevede che se eseguite sulla ClasseVirtuale vengano propagate sul Registro.

In aggiunta a questo, orphanRemoval=true significa che se un Registro viene rimosso dalla relazione con la sua Classe, Hibernate deve eliminarlo anche dal database. Queste cose sono coerenti con la composizione: un Registro senza la propria Classe non ha senso.

@JoinColumn(name = “registro_id”) significa che nella tabella ClasseVirtuale è memorizzata la foreign key registro_id verso Registro. Con unique=true evitiamo che due classi virtuali facciano riferimento allo stesso Registro.

Lato Registro:

```
@OneToOne(
    mappedBy = "registro",
    fetch = FetchType.EAGER
)
private ClasseVirtuale classeVirtuale;
```

Con mappedBy=”registro” diciamo che la relazione è gestita dall’attributo registro dell’altro lato della relazione, cioè di ClasseVirtuale.

Per quanto riguarda le relazioni di composizione tra Registro e Compito, Lezione e Valutazione possiamo analizzarne la “traduzione” in annotazioni jakarta solo in un caso, perché sono pressoché identiche.

Esempio: associazione uno a molti unidirezionale tra Registro e Compito. Un compito è assegnato a un solo Registro, un Registro contiene molti compiti. L’associazione è unidirezionale, infatti in Registro manteniamo una lista di compiti, mentre in Compito non manteniamo un attributo Registro.

Lato Registro:



```
@OneToMany(
   cascade = CascadeType.ALL,
   orphanRemoval = true,
   fetch = FetchType.EAGER
)
@JoinColumn(name = "registro_id")
private List<Compito> compitiRegistrati
```

Qui c'è un problema. In una relazione uno a molti la JoinColumn() si mette in genere nel lato @ManyToOne sull'attributo che rappresenta il lato 1. In questo caso però l'associazione è unidirezionale, quindi Compito non mantiene un attributo registro. Però di certo non possiamo mettere la foreign key nella tabella di Registro perché non possiamo prevedere più valori per tale colonna associati a tutti i compiti collegati al registro. A livello di database quindi si sceglie comunque per rappresentare la relazione tra le due classi di mettere una colonna nella tabella di Compito che faccia da foreign key verso Registro. Non potendo mettere la @JoinColumn dal lato di Compito non essendoci l'attributo registro, lo mettiamo sull'attributo compitiRegistrati dal lato di Registro perché tanto Hibernate è in grado comunque di capire, guardando la struttura della relazione, che la foreign key, anche se la @JoinColumn(name = "registro_id") è specificata dal lato di Registro, va messa dal lato di Compito.

cascade=CascadeType.ALL indica che tutte le operazioni su Registro sono propagate anche sui Compiti associati. orphanRemoval=true indica che se togliamo un compito dalla lista del registro quel compito può essere eliminato dal database. Questi due parametri quindi, come prima tra ClasseVirtuale e Registro, mi permettono di realizzare la semantica della composizione.

5.3 Package Controller

Classi:

- ControllerClassi
- ControllerUtenti
- ControllerRegistro

Librerie:

- java.util
- java.time

5.4 Package Boundary

Classi:

- SchermataClasse
- SchermataDocente
- SchermataIniziale
- SchermataRegistraValutazione
- SchermataRegistro

- SchermataSelezioneRegistraVoce
- SchermataStudente
- SchermataValutazioniStudenti
- SchermataVisualizzaInfo

Librerie:

- javax.swing
- java.awt
- java.util
- java.time
- com.formdev.flatlaf
- com.intellij.uiDesigner

5.5 Package DTO

L'introduzione di tale package, estraneo al pattern BCED, nasce dall'esigenza di mostrare sulla GUI collezioni di elementi.

Da questo punto di vista, il problema principale è proprio quello che, qualora una determinata chiamata a funzione restituisse alla GUI un elenco di Entity, questa, per poterlo visualizzare correttamente a video, dovrebbe conoscere di fatto la struttura interna di tale classe Entity, ma ciò porterebbe con sé un accoppiamento troppo elevato e quindi una chiara violazione dei vincoli del pattern a livelli adottato.

Si introduce allora il concetto di **Data Transfer Object** (**DTO**), un oggetto in grado di trasportare dati tra processi (nel caso in oggetto tra livelli). Le classi DTO hanno tipicamente una struttura che rispecchia quella dell'Entity di cui vanno a supporto, in particolare gli attributi coincidono con quelli dell'Entity che si intendono visualizzare a schermo.

Ecco le classi DTO che abbiamo inserito nel package DTO. Il nome è indicativo delle classi dell'Entity ad esse associate:

- DocenteDTO
- InfoClasseDTO
- InfoValutazioneDTO
- StudenteDTO

5.6 Package Exceptions

Nel package Exceptions abbiamo definito le eccezioni:

- ClasseEsistenteException
- ClasseInesistenteException
- GiaIscrittoException

Tutte e tre queste eccezioni estendono Exception, quindi sono delle checked exceptions, per cui il metodo che le solleva deve dichiararle con throws e chi lo invoca dovrà gestirle con un try/catch (oppure propagarle). In particolare:

- ClasseEsistenteException è usata nel caso d'uso "Creazione classe virtuale". Il metodo creaClasse() di ControllerClassi la solleva se il docente ha già utilizzato lo stesso nome



per un'altra delle sue classi. SchermataDocente gestisce l'eccezione con un try-catch comunicando al Docente tramite la GUI che il nome è già in uso per un'altra classe.

- ClasseInesistenteException è usata nel caso d'uso "Iscrizione studente tramite codice classe". Il metodo iscrizioneStudenteTramiteCodiceClasse() la solleva se non esiste una classe associata al codice inserito dallo studente. SchermataStudente la intercetta e la gestisce con un try-catch comunicando allo Studente tramite la GUI che la classe avente quel codice non esiste.
- GiaIscrittoException è usata nel caso d'uso "Iscrizione studente tramite codice classe". Il metodo iscrizioneStudenteTramiteCodiceClasse() la solleva se lo studente è già iscritto alla classe. SchermataStudente gestisce l'eccezione con un try-catch comunicando allo Studente tramite la GUI che è già iscritto alla classe avente quel codice e quindi non può iscriversi di nuovo.

5.7 Package Comunicazione

In questo package inseriamo come unica classe GestoreNotifiche. Se conoscessimo il sistema di notifiche esterno utilizzato avremmo ovviamente la possibilità di aggiungere altri elementi all'interno del package.

5.8 Artefatti necessari

Per l'esecuzione dell'applicazione, senza ricorrere a un ambiente di sviluppo integrato (IDE), sono necessari i seguenti componenti:

1. Java Runtime: è necessario disporre di un ambiente di esecuzione Java compatibile con la versione 23, utilizzata per la compilazione del progetto.
2. Database MySQL: è necessaria l'installazione e l'esecuzione di un server MySQL (con utente "root" e password "FASOLINO2026"), utilizzato per la memorizzazione persistente dei dati. Il collegamento al database avviene mediante JDBC, con i parametri definiti nel file META-INF/persistence.xml. Il database utilizzato è denominato SistemaRegistroElettronico_db.
3. File eseguibile JAR: il programma viene distribuito attraverso il file SistemaRegistroElettronico.jar, contenente le classi Java compilate (file .class), le risorse applicative e le librerie necessarie al funzionamento. Il file JAR contiene inoltre il riferimento alla classe principale it.unina.registroelettronico.Boundary.Swing.Main, che costituisce il punto di avvio dell'applicazione.
4. Librerie e dipendenze: il progetto utilizza Hibernate ORM 6.6.50.Final per la persistenza dei dati, MySQL Connector/J 9.4.0 per la connessione al database, FlatLaf Extras 3.7.2 per gli elementi grafici e IntelliJ Forms Runtime 7.0.3 per il supporto ai componenti dell'interfaccia. Tali librerie, insieme alle relative dipendenze necessarie, sono incluse nell'archivio JAR distribuito e non richiedono un'installazione separata.
5. File di configurazione della persistenza: il file META-INF/persistence.xml, incluso nell'archivio JAR, contiene le informazioni necessarie alla configurazione della connessione MySQL e alla gestione della persistenza tramite Hibernate.

Per avviare il programma è necessario assicurarsi che il server MySQL sia attivo, che le credenziali di accesso siano configurate correttamente e che Java sia installato. Non è necessaria l'installazione di un IDE come Eclipse o IntelliJ IDEA, né di Maven, in quanto il programma viene distribuito già compilato con le dipendenze necessarie all'esecuzione.



5.9 Diagramma di Deployment

L'intera applicazione è eseguita sul PC Utente, sul quale sono presenti sia il sistema applicativo sia il database.

È configurato, innanzitutto, su tale nodo unico l'ambiente di esecuzione, Java Runtime 23, necessario per l'esecuzione dell'applicazione Java. In tale ambiente viene distribuito l'artefatto principale SistemaRegistroElettronico.jar, che rappresenta l'applicazione eseguibile.

La persistenza dei dati è affidata a un MySQL Server, anch'esso installato sul PC dell'utente. Al suo interno è presente il database SistemaRegistroElettronico_db, nel quale vengono memorizzati i dati gestiti dall'applicazione. La comunicazione tra l'applicazione Java e il server MySQL avviene tramite JDBC, utilizzando il protocollo TCP/IP e la porta standard di MySQL 3306. L'indicazione localhost:3306 evidenzia che il database viene eseguito sulla stessa macchina sulla quale è in esecuzione l'applicazione.



6. Testing

6.1 Test Strutturale

6.1.1 Complessità ciclomatica

Si intende costruire il Control Flow Graph per due dei metodi delle classi implementate e mostrare il calcolo del numero ciclomatico e i percorsi linearmente indipendenti.

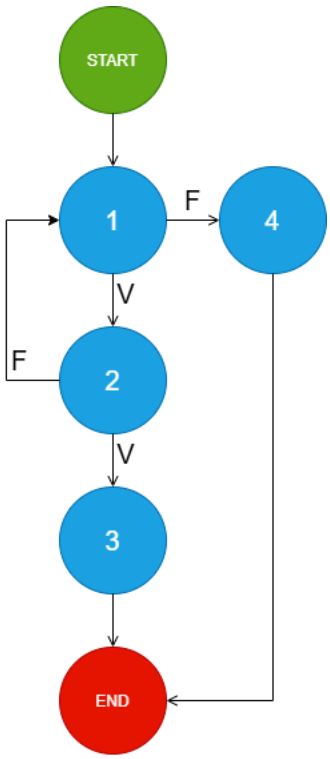
6.1.1.1 verificaUnivocitaNome – Docente

```
public boolean verificaUnivocitaNome(String nome) {

    for(ClasseVirtuale c:classiGestite){ (1)
        if(c.getNome().equals(nome)) (2)
            return false; (3)
    }

    return true; (4)

}
```



NUMERO CICLOMATICO:

- numero di regioni chiuse del grafo + 1 = 2+1 = 3
- numero di nodi predicati (nodo 1 e nodo 2) +1 = 2+1= 3
- # archi – # nodi + 2 = (7 - 6) + 2= 3

CAMMINI LINEARMENTE INDIPENDENTI:

1) 1-4
2) 1-2-3
3) 1-2-1-4

CASI di TEST

TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/ FAIL)
TC_1	1-4	Il docente non gestisce alcuna classe	La condizione al punto 1 del CFG è false	nome= "3A Informatica"	classiGestite vuota	true	true	**PASS**
TC_2	1-2-3	Esiste già una classe con lo stesso nome, trovata alla prima iterazione	La condizione al punto 1 del CFG è true, la condizione al punto 2 del CFG è true	nome= "3A Informatica	Il docente gestisce già una classe chiamata "3A Informatica"	false	false	**PASS**
TC_3	1-2-1-4	Il docente gestisce una sola classe, ma con	La condizione al punto 1 del CFG è true, la	nome= "3A Informatica	Il docente gestisce una sola classe chiamata "4B Matematica"	true	true	**PASS**

		nome differente	condizione al punto 2 è false e alla successiva verifica la condizione al punto 1 è false					

6.1.1.2 cercaValutazioniStudente – Registro

```
public List<List<String>> cercaValutazioniStudente(String emailStudente) {

    List<List<String>> valStudente=new ArrayList<>(); (1)

    for(Valutazione v:valutazioniRegistrate){ (2)

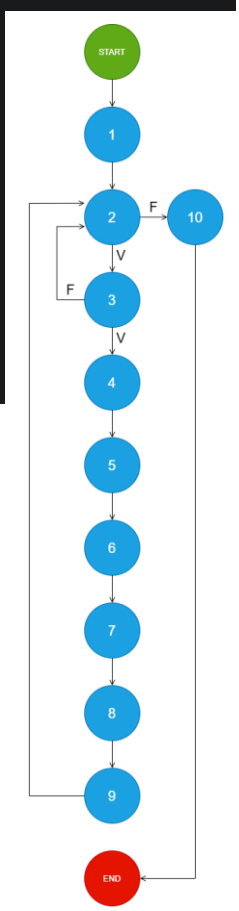

if(v.getStudente().getIndirizzoEmailIstituzionale().equals(emailStudente)){ (3)
            List<String> val=new ArrayList<>(); (4)

            val.add(v.getData().toString()); (5)
            val.add(v.getTipologia().toString()); (6)
            val.add(Float.toString(v.getVoto())); (7)
            val.add(v.getNotaDescrittiva()); (8)

            valStudente.add(val); (9)

        }

    }
    return valStudente; (10)
}
```



Abbiamo considerato un CFG non ridotto.

NUMERO CICLOMATICO:

- numero di regioni chiuse del grafo + 1 = 2+1 = 3
- numero di nodi predicati (nodo 2 e nodo 3) +1 = 2+1= 3
- # archi – # nodi + 2 = (13 - 12) + 2= 3

CAMMINI LINEARMENTE INDIPENDENTI:

1) 1-2-10
2) 1-2-3-4-5-6-7-8-9-2-10
3) 1-2-3-2-10

CASI di TEST

TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/ FAIL)
TC_1	1-2-10	Il registro non contiene valutazioni	La condizione al punto 2 del CFG è false	emailStudente = "studente.a@test.it"	valutazioniRegistrate vuota	Lista vuota []	Lista vuota []	**PASS**
TC_2	1-2-3-4-5-6-7-8-9-2-10	Il registro contiene una sola valutazione dello studente cercato	La condizione al punto 2 del CFG è true, quella al punto 3 è true e successivamente quella al punto 2 è false	emailStudente = "studente.a@test.it"	È presente una valutazione associata a studente.a@test.it	Lista contenente i dati della valutazione	Lista contenente i dati della valutazione	**PASS**
TC_3	1-2-3-2-10	Il registro contiene una valutazione appartenente a uno studente diverso	La condizione al punto 2 del CFG è true, quella al punto 3 è false e successivamente quella al punto 2 è false	emailStudente = "studente.a@test.it"	È presente una valutazione associata a studente.b@test.it	Lista vuota []	Lista vuota []	**PASS**

6.2 JUnit – Test di Unità

I test di unità realizzati con JUnit sono disponibili nella cartella test del progetto Java, a cui rimandiamo per questa sezione. (Evitiamo screenshot per comodità e leggibilità).

6.3 Test funzionale

Segue una descrizione in forma tabellare dei risultati dell'esecuzione dei test funzionali precedentemente pianificati.

TEST SUITE- CREAZIONE CLASSE VIRTUALE									
Test Case ID	**Descrizione**	**Classi di equivalenza coperte**	**Pre-condizioni**	**Input**	**Output attesi**	**Post-condizioni attese**	**Output ottenuti**	**Post-condizioni ottenute**	**Esito (FAIL, PASS)**
1	Nome della classe valido	CV1: Stringa di caratteri di lunghezza >0 e <=50	Docente autenticato; **il docente non possiede una classe chiamata "3A Informatica"**	Nome = "3A Informatica"	La classe viene creata; il sistema assegna un codice univoco e la mostra nell'elenco delle classi del docente	La nuova classe è persistita e associata al docente; il codice generato non è disponibile per altre classi	La classe viene creata; il sistema assegna un codice univoco e la mostra nell'elenco delle classi del docente	La nuova classe è persistita e associata al docente; il codice generato non è disponibile per altre classi	**PASS**
2	Nome già utilizzato	CV1: Stringa di caratteri di lunghezza >0 e <=50	Docente autenticato; **il docente possiede già una classe chiamata "3A Informatica"**	Nome = "3A Informatica"	Il sistema vieta la creazione e comunica che il nome è già utilizzato	Nessuna nuova classe viene creata; lo stato delle classi del docente rimane invariato	Il sistema vieta la creazione e comunica che il nome è già utilizzato	Nessuna nuova classe viene creata; lo stato delle classi del docente rimane invariato	**PASS**
3	Nome troppo lungo	CNV1: Stringa di caratteri di lunghezza >50	Docente autenticato	Una stringa di 51 caratteri	Il sistema vieta la creazione e comunica che il nome supera la	Nessuna nuova classe viene creata; lo stato delle classi del	Il sistema vieta la creazione e comunica che il nome supera la	Nessuna nuova classe viene creata; lo stato delle classi del	**PASS**



					lunghezza massima consentita	docente rimane invariato	lunghezza massima consentita	docente rimane invariato	
4	Nome non inserito	CNV2: Stringa vuota	Docente autenticato	Nome = ""	Il sistema vieta la creazione e comunica che è necessario inserire il nome della classe	Nessuna nuova classe viene creata; lo stato delle classi del docente rimane invariato	Il sistema vieta la creazione e comunica che è necessario inserire il nome della classe	Nessuna nuova classe viene creata; lo stato delle classi del docente rimane invariato	**PASS**

TEST SUITE- ISCRIZIONE TRAMITE CODICE CLASSE									
Test Case ID	**Descrizione**	**Classi di equivalenza coperte**	**Pre-condizioni**	**Input**	**Output attesi**	**Post-condizioni attese**	**Output ottenuti**	**Post-condizioni ottenute**	**Esito (FAIL, PASS)**
1	Codice valido	CV1: Stringa di caratteri di lunghezza =10, con prefisso "CLS-"	Studente autenticato; **esiste una classe con codice "CLS-A23BC6"; lo studente non è iscritto a quella classe**	Codice= "CLS-A23BC6"	Il sistema conferma l'iscrizione e mostra la classe nell'elenco delle classi dello studente	Lo studente risulta iscritto alla classe	Il sistema conferma l'iscrizione e mostra la classe nell'elenco delle classi dello studente	Lo studente risulta iscritto alla classe	**PASS**
2	Codice inesistente	CV1: Stringa di caratteri di lunghezza =10, con prefisso "CLS-"	Studente autenticato; **nessuna classe possiede il codice "CLS-999999"**	Codice= "CLS-999999"	Il sistema vieta l'operazione e comunica che il codice è inesistente	Nessuna nuova iscrizione	Il sistema vieta l'operazione e comunica che il codice è inesistente	Nessuna nuova iscrizione	**PASS**
3	Studente già iscritto	CV1: Stringa di caratteri di lunghezza =10,	Studente autenticato; **esiste la classe**	Codice= "CLS-123456"	Il sistema vieta l'operazione e comunica	Nessuna nuova iscrizione	Il sistema vieta l'operazione e	Nessuna nuova iscrizione	**PASS**

		con prefisso "CLS-"	**"CLS-123456"; lo studente è già iscritto**		che lo studente è già iscritto		comunica che lo studente è già iscritto		
4	Codice con lunghezza errata	CNV1: Stringa di caratteri non vuota di lunghezza diversa da 10	Studente autenticato	Codice= "CLS-12345"	Il sistema vieta l'operazione e comunica che il codice deve essere composto da 10 caratteri	Nessuna nuova iscrizione	Il sistema vieta l'operazione e comunica che il codice deve essere composto da 10 caratteri	Nessuna nuova iscrizione	**PASS**
5	Prefisso del codice errato	CNV2: Stringa di lunghezza =10 che non inizia con "CLS-"	Studente autenticato	Codice= "ABC-123456"	Il sistema vieta l'operazione e comunica che il codice deve iniziare con CLS-	Nessuna nuova iscrizione	Il sistema vieta l'operazione e comunica che il codice deve iniziare con CLS-	Nessuna nuova iscrizione	**PASS**
6	Codice non inserito	CNV3: Stringa vuota	Studente autenticato	Codice= ""	Il sistema vieta l'operazione e comunica che è necessario inserire il codice della classe	Nessuna nuova iscrizione	Il sistema vieta l'operazione e comunica che è necessario inserire il codice della classe	Nessuna nuova iscrizione	**PASS**

TEST SUITE- REGISTRA VALUTAZIONE									
Test Case ID	**Descrizione**	**Classi di equivalenza coperte**	**Pre-condizioni**	**Input**	**Output attesi**	**Post-condizioni attese**	**Output ottenuti**	**Post-condizioni ottenute**	**Esito (FAIL, PASS)**
1	Tutti gli input validi	T-CV1: Tipologia = INTERROGAZIONE; V-CV1: voto selezionato; D-CV1: data valida; N-CV1: nota 0–200; S-CV1: studente selezionato	Docente autenticato; classe con almeno Studente A iscritto	Studente=A; Tipologia=INTERROGAZIONE; Data=03/10/2026; Voto=7.25; Nota="Buona prova"	Il Sistema comunica l'avvenuta registrazione della valutazione	La valutazione è persistita per Studente A; viene avviato l'invio della notifica	Il Sistema comunica l'avvenuta registrazione della valutazione	La valutazione è persistita per Studente A; viene avviato l'invio della notifica	**PASS**
2	Tipologia non selezionata	**T-CNV1: Tipologia non selezionata**; tutti gli altri input validi (V-CV1, D-CV1, N-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=non selezionata; Data=03/10/2026; Voto=7.25; Nota="Buona prova"	Il Sistema vieta l'operazione e comunica che è necessario selezionare la tipologia	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che è necessario selezionare la tipologia	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**
3	Voto non selezionato	T-CV2: Tipologia=PROVA_SCRITTA; **V-CNV1: voto non selezionato**; tutti gli altri input validi (D-CV1, N-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=PROVA_SCRITTA; Data=03/10/2026; Voto=non selezionato; Nota="Buona prova"	Il Sistema vieta l'operazione e comunica che è necessario selezionare il voto	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che è necessario selezionare il voto	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**
4	Data successiva alla data corrente	**D-CNV1: Data futura**; tutti gli altri input validi (T-CV1, V-CV1, N-CV1, S-CV1)	Docente autenticato; riferimento temporale corrente: 05/10/2026; classe	Studente=A; Tipologia=INTERROGAZIONE; Data=25/12/2027; Voto=6.5; Nota=""	Il Sistema vieta l'operazione e comunica che la data non può essere successiva a	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che la data non può essere successiva	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**

			con almeno Studente A		quella corrente		a quella corrente		
5	Data corrispondente a domenica	**D-CNV2: Data non futura il cui giorno è domenica**; tutti gli altri input validi (T-CV2, V-CV1, N-CV1. S-CV1)	Docente autenticato: riferimento temporale corrente: 05/10/2026; classe con almeno Studente A	Studente=A; Tipologia=PROVA_SCRITTA; Data=04/10/2026; Voto=8; Nota=""	Il Sistema vieta l'operazione e comunica che non è possibile selezionare una domenica	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che non è possibile selezionare una domenica	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**
6	Data non selezionata	**D-CNV3: Data non selezionata**; tutti gli altri input validi (T-CV1, V-CV1, N-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=INTERROGAZIONE; Data=non selezionata; Voto=5.75; Nota="Da migliorare"	Il Sistema vieta l'operazione e comunica che è necessario selezionare una data	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che è necessario selezionare una data	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**
7	Nota troppo lunga	**N-CNV1: Nota di lunghezza >200**; tutti gli altri input validi (T-CV2, V-CV1, D-CV1, S-CV1)	Docente autenticato; classe con almeno Studente A	Studente=A; Tipologia=PROVA_SCRITTA; Data=03/10/2026; Voto=9; Nota=stringa di 201 caratteri	Il Sistema vieta l'operazione e comunica che la nota può contenere al massimo 200 caratteri	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che la nota può contenere al massimo 200 caratteri	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**
8	Studente non selezionato	**S-CNV1: Studente non selezionato**; tutti gli altri input validi (T-CV1, D-CV1, V-CV1, N-CV1)	Docente autenticato; la classe contiene almeno uno studente	Studente=non selezionato; Tipologia=INTERROGAZIONE; Data=03/10/2026; Voto=6.25; Nota=""	Il Sistema vieta l'operazione e comunica che è necessario selezionare uno studente	Nessuna valutazione viene registrata; nessuna notifica viene inviata	Il Sistema vieta l'operazione e comunica che è necessario selezionare uno studente	Nessuna valutazione viene registrata; nessuna notifica viene inviata	**PASS**